SQL ANALYTICS CASE STUDY · MYSQL 8

Marketplace Contribution & Discount Analytics

A MySQL case study on a single marketplace transactions table — a monthly commercial performance bridge, customer concentration, discount dependency, acquisition cohorts and a discount-reduction break-even — answered with CTEs, window functions and CASE-driven segmentation.

561,318	447,907	390,593	226
TOTAL GMV	DELIVERED GMV	CONTRIBUTION PROXY	CUSTOMERS

DATABASE ENGINE

MySQL 8.0

InnoDB · window functions required

SCHEMA

case_study_1

1 table · 18 columns · 5 analytical questions

SOURCE DATA

marketplace_transactions

12 months of 2025 orders · 226 purchasing customers

CONCEPTS COVERED

CTEs · Windows · CASE

LAG(), NTILE(), PERCENT_RANK(), NULLIF, UNION ALL

WHAT THIS DOCUMENT CONTAINS

Every query in the case study is reproduced here in full, with the result grids exactly as the executed script printed them, five analytical questions with charts and key insights, a code-review section documenting the defects found in the original script with a correction for each, and a short results and roadmap section.

CONTENTS

Table of Contents

01 **Project Overview** 3

Scope, objectives and the analytical skills the case study demonstrates.

02 **Table Architecture & Metric Definitions** 5

The single source table, its eighteen columns and how every metric is built.

03 **Data Preview & Setup** 7

Database selection, a ten-row sample and column inspection.

04 **Commercial Performance Analytics** 9

Questions 1–2: the monthly commercial bridge and customer concentration.

05 **Discount, Cohort & Scenario Analytics** 20

Questions 3–5: discount dependency, acquisition cohorts and discount-reduction break-even.

06 **Code Review & Corrections** 39

Seven logic and data defects, six design observations, and the fix for each.

07 **Results, Setup & Roadmap** 45

Headline results, how to run the case study and future enhancements.

SECTION 01

Project Overview

A single-table MySQL case study that rebuilds a marketplace's monthly commercial performance bridge, then layers customer, discount, cohort and scenario analytics on top of it.

The **case_study_1** database holds one table, `marketplace_transactions`, in which every row is an order line of a multi-seller marketplace: the product category, the seller, the customer and how that customer was acquired, the order status, list and selling prices, the discount, refund, payment fee and fulfilment cost, and the seller commission rate. On this one table sit five analytical questions that a commercial finance or growth team would ask in sequence: how GMV converts to contribution, who generates it, how much of it is bought with discount, which acquisition cohorts hold their value, and how much volume could be lost before a discount cut stops paying for itself.

BASE TABLE	COLUMNS	ANALYTICAL QUESTIONS
1	18	5
marketplace_transactions	text · int · double	32 queries, CTE-based

MONTHS ANALYSED	PURCHASING CUSTOMERS	PRODUCT CATEGORIES
12	226	5
January – December 2025	at least one delivered order	5 channels · 3 segments

Objectives

- **Reconstruct the bridge.** Monthly GMV split into delivered, cancelled and returned, with discounts, refunds, seller commission, payment fees, fulfilment cost and a contribution proxy, compared month over month.
- **Measure concentration.** Share of delivered GMV and contribution from the top 1%, 5% and 10% of customers, repeat-purchase participation, and whether the largest customers are also the most profitable.
- **Test discount dependency.** Discount rates by category and acquisition channel, contribution per delivered order, segment economics and the weakest post-discount transactions.
- **Judge acquisition quality.** Acquisition-month cohorts compared on total and per-order contribution and repeat-customer rate.
- **Size a discount cut.** A 10% reduction in discount expense and the delivered-GMV decline that would cancel its benefit, by category and month.
- **Review its own work.** Section 06 audits the script against the data and documents each defect with a correction.

Question Map

Question	Business question	Techniques	Queries
Q1 Commercial bridge	How does monthly GMV convert to contribution, and where does growth in one diverge from the other?	conditional SUM, LAG(), growth CTE	11
Q2 Customer concentration	How concentrated are delivered GMV and contribution, and are the biggest customers profitable?	NTILE(100), PERCENT_RANK(), UNION ALL	4
Q3 Discount dependency	Which categories, channels and segments depend on discount, and what does it do to contribution per order?	NULLIF ratios, CASE risk label, segment roll-ups	9
Q4 Acquisition cohorts	Which acquisition months produce the most contribution and the best repeat behaviour?	derived table with MIN() per customer	5
Q5 Break-even decline	How far can delivered GMV fall before a 10% discount cut stops paying?	ratio of saving to contribution, monthly grain	3

Technology and Techniques

Area	Applied in this case study
Session	CREATE DATABASE, USE, SHOW TABLES, DESCRIBE
Date logic	STR_TO_DATE on text dates, DATE_FORMAT(..., '%Y-%m') for monthly and cohort buckets
Aggregation	COUNT(DISTINCT ...), conditional SUM(CASE WHEN order_status = ...) for status-specific metrics
CTE pipelines	monthly → growth; customer_totals → ranked_customers → concentration; category_metrics → final labels
Window functions	LAG() for month-over-month growth, NTILE(100) for percentile bands, PERCENT_RANK() for GMV percentile
Null-safe maths	NULLIF in every ratio so an empty denominator returns NULL instead of an error
Segmentation	CASE expressions that turn numbers into labels: negative_contribution, commercial_risk
Set operations and subqueries	UNION ALL for the three concentration bands; scalar subqueries for totals; derived tables for one-row-per-customer cohorts

SECTION 02

Table Architecture & Metric Definitions

One table, eighteen columns, four jobs: identify the order line, describe the customer, price it and cost it.

Every analytical question is answered from `marketplace_transactions` alone; there are no joins. Columns fall into six groups. All date columns are stored as **TEXT**, so every date operation first needs `STR_TO_DATE`. Customer attributes (acquisition date, channel, segment) are repeated on every line of the customer rather than held in a customer table, a design that matters for the cohort and segment questions (see Section 06).

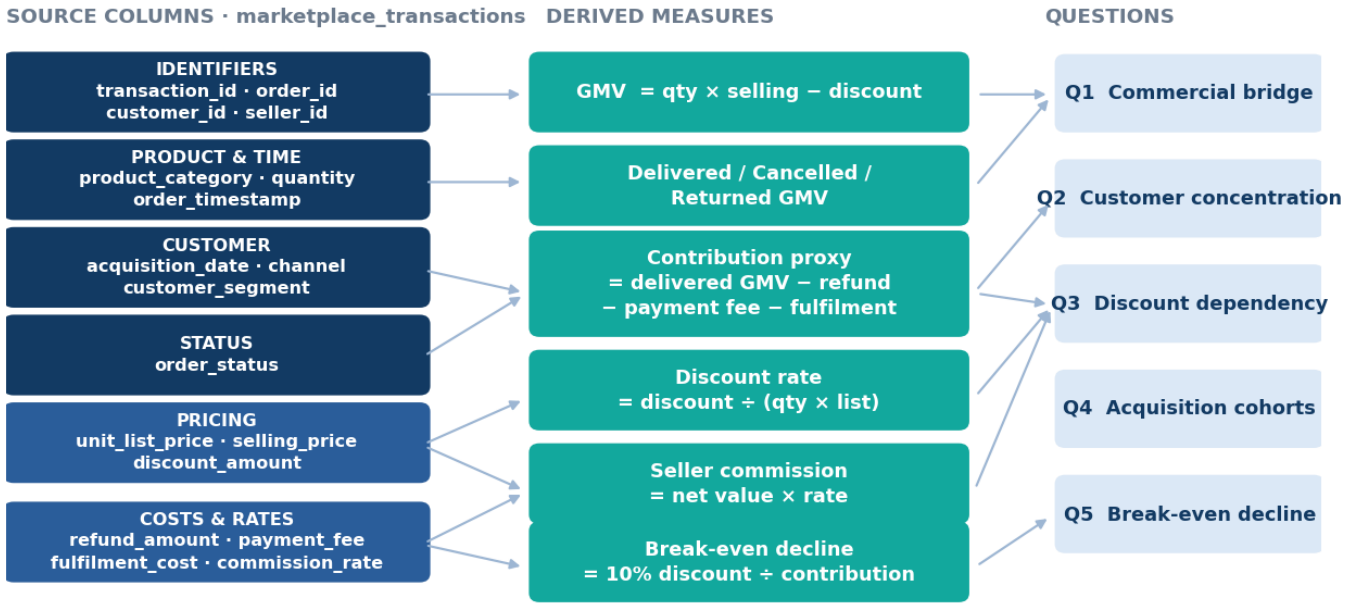


Figure 2.1 — Source column groups, the derived measures built from them, and the questions that use them.

Column Dictionary

Column	Type	Group	Meaning
transaction_id	text	Identifiers	Transaction (order-line) identifier, T00001 onwards
order_id	text	Identifiers	Order identifier, O00xxx
customer_id	text	Identifiers	Customer identifier, C0xxx
seller_id	text	Identifiers	Seller identifier, S0xx
product_category	text	Product & time	Beauty, Electronics, Fashion, Home & Kitchen, Sports
order_timestamp	text	Product & time	Order date, stored as text
quantity	int	Product & time	Units on the line
customer_acquisition_date	text	Customer	Date the customer was acquired, stored as text
acquisition_channel	text	Customer	Affiliate, Organic, Paid Search, Referral, Social
customer_segment	text	Customer	Standard, Premium, High Value
order_status	text	Status	Delivered, Cancelled, Returned and at least one further status
unit_list_price	double	Pricing	List price per unit
unit_selling_price	double	Pricing	Selling price per unit
discount_amount	double	Pricing	Discount on the line
refund_amount	double	Costs & rates	Refund paid on the line
payment_fee	double	Costs & rates	Payment processing fee
fulfilment_cost	double	Costs & rates	Fulfilment cost of the line
seller_commission_rate	double	Costs & rates	Commission rate applied to the line

Metric Definitions as Implemented

These are the definitions the script uses, quoted exactly; Section 06 explains why two of them (GMV and the refund leg of the contribution proxy) should be revised.

Metric	Definition in the script	Rows used
GMV	`quantity * unit_selling_price - discount_amount`	all statuses
Delivered / Cancelled / Returned GMV	same expression, filtered on order_status	one status
Seller commission	`GMV * seller_commission_rate`	Delivered
Contribution proxy	`GMV - refund_amount - payment_fee - fulfilment_cost`	Delivered
Discount rate	`SUM(discount_amount) / SUM(quantity * unit_list_price)`	all, or Delivered
Refund rate	`SUM(refund_amount) / delivered GMV`	Delivered
Repeat customer	at least two distinct delivered orders	Delivered
Break-even GMV decline	`10% × delivered discount ÷ contribution proxy`	Delivered

SECTION 03

Data Preview & Setup

Steps 1–3: selecting the database, previewing ten rows and inspecting the column list.

The source captures begin after the table had been loaded; the import step itself is not part of the case study.

Step 1 — Create and select the database

STEP 1 · CREATE AND SELECT THE DATABASE

```
CREATE DATABASE case_study_1;

USE case_study_1;

SHOW TABLES;
```

Step 2 — Preview ten rows

STEP 2 · PREVIEW TEN ROWS

```
SELECT *
FROM marketplace_transactions
LIMIT 10;
```

OUTPUT — 10 ROWS (IDENTIFIERS, CUSTOMER AND STATUS COLUMNS)

transaction_id	order_id	customer_id	seller_id	product_category	order_timestamp	customer_acquisition_date	acquisition_channel	order_status	quantity
T00001	O00411	C0094	S029	Sports	14-08-2025	06-07-2025	Affiliate	Delivered	2
T00002	O00614	C0006	S027	Home & Kitchen	12-02-2025	19-05-2023	Social	Delivered	1
T00003	O00040	C0097	S030	Sports	30-09-2025	29-01-2023	Paid Search	Returned	1
T00004	O00062	C0203	S002	Electronics	20-09-2025	25-08-2023	Referral	Delivered	1
T00005	O00163	C0048	S023	Electronics	07-04-2025	09-04-2024	Social	Delivered	1
T00006	O00640	C0234	S049	Home & Kitchen	20-12-2025	29-05-2023	Organic	Cancelled	2
T00007	O00203	C0086	S069	Beauty	17-12-2025	23-06-2024	Social	Delivered	1
T00008	O00060	C0073	S023	Beauty	10-03-2025	24-04-2024	Social	Cancelled	1
T00009	O00256	C0167	S005	Sports	28-03-2025	24-03-2024	Affiliate	Delivered	1
T00010	O00385	C0104	S023	Beauty	19-04-2025	30-10-2024	Social	Delivered	1

OUTPUT — 10 ROWS (PRICING, COST AND SEGMENT COLUMNS)

transaction_id	unit_list_price	unit_selling_price	discount_amount	refund_amount	payment_fee	fulfilment_cost	seller_commission_rate	customer_segment
T00001	412.77	393.72	38.09	0	21.24	46.04	0.122	High Value
T00002	437	367.67	69.33	0	6.92	67.84	0.153	Premium
T00003	321.99	308.42	13.57	230.58	6.46	101.56	0.079	High Value
T00004	135.62	119.14	16.48	0	2.88	36.17	0.122	Premium
T00005	1064.24	945.68	118.56	0	20.99	96.76	0.146	Standard
T00006	1510.14	1176.15	667.98	0	42.13	66.82	0.149	Premium
T00007	469.51	442.76	26.75	0	14.1	71.13	0.107	Standard
T00008	141.29	131.44	9.85	0	3.12	85.58	0.081	Standard
T00009	895.32	736.66	158.66	0	17.2	99.32	0.157	Standard
T00010	393.45	346.2	47.25	0	11.61	15.75	0.145	Standard

Step 3 — Check the column names

STEP 3 · CHECK THE COLUMN NAMES

```
-- check columns name
DESCRIBE marketplace_transactions;
```

OUTPUT — 18 ROWS

Field	Type	Null	Key	Default	Extra
transaction_id	text	YES		NULL	
order_id	text	YES		NULL	
customer_id	text	YES		NULL	
seller_id	text	YES		NULL	
product_category	text	YES		NULL	
order_timestamp	text	YES		NULL	
quantity	int	YES		NULL	
customer_acquisition_date	text	YES		NULL	
acquisition_channel	text	YES		NULL	
customer_segment	text	YES		NULL	
order_status	text	YES		NULL	
unit_list_price	double	YES		NULL	
unit_selling_price	double	YES		NULL	
discount_amount	double	YES		NULL	
refund_amount	double	YES		NULL	
payment_fee	double	YES		NULL	
fulfilment_cost	double	YES		NULL	
seller_commission_rate	double	YES		NULL	

WHAT THE PREVIEW ALREADY SHOWS

Discount is already inside the selling price. On all ten sample rows, `discount_amount` equals `quantity × (unit_list_price − unit_selling_price)` to within one cent (for example T00001: 2 × (412.77 − 393.72) = 38.10 against 38.09). The GMV expression used throughout the script subtracts `discount_amount` again; Defect 1 in Section 06 quantifies the effect. **Refunds sit only on returned orders:** the one non-zero refund (230.58) is on T00003, which is Returned. **Dates:** the capture shows dd-mm-yyyy, a spreadsheet display format; the queries parse the text as `%Y-%m-%d` and return valid months, so the stored text must be ISO.

SECTION 04

Commercial Performance Analytics

Questions 1–2: the monthly commercial bridge from GMV to contribution, and how concentrated that contribution is across customers.

TOTAL GMV	DELIVERED GMV	CONTRIBUTION PROXY	PURCHASING CUSTOMERS	TOP-10% SHARE
561,318	447,907	390,593	226	34.1%
12 months · all statuses	79.8% of total GMV	87.2% of delivered GMV	≥1 delivered order	of delivered GMV · 30 customers

HOW THE OUTPUTS WERE PRODUCED

The result grids in this document are transcribed from the MySQL result captures in the source case study; no query was re-run while preparing the document, so every figure is exactly as the author's run printed it. ◆ Derived figures are simple arithmetic on the printed outputs (for example summing twelve monthly values); they were not produced by a query. Several source captures show only the first 29 rows of a longer result; each table says so.

Question 1 — Reconstruct the commercial performance bridge

Build the monthly bridge from GMV to contribution: GMV, delivered, cancelled and returned GMV, discounts, refunds, seller commission, payment fees, fulfilment costs and the contribution proxy; then compare month-over-month growth in order count, GMV, delivered GMV and contribution proxy, and identify months where GMV growth is positive but contribution growth is negative. The script computes each component in its own query (Steps 1–10) and then combines them in one CTE pipeline (Step 11).

Step 1 — Monthly GMV and order count

QUESTION 1 · STEP 1 — MONTHLY GMV AND ORDER COUNT

```sql
-- Monthly GMV and order count
SELECT
  DATE_FORMAT(
    STR_TO_DATE(order_timestamp, '%Y-%m-%d'),
    '%Y-%m'
  ) AS order_month,
  COUNT(DISTINCT order_id) AS order_count,
  ROUND(
    SUM(quantity * unit_selling_price - discount_amount),
    2
  ) AS total_gmv
FROM marketplace_transactions
GROUP BY order_month
ORDER BY order_month;
```

Step 2 — Monthly delivered GMV

QUESTION 1 · STEP 2 — MONTHLY DELIVERED GMV

```sql
-- Monthly delivered GMV
SELECT
  DATE_FORMAT(
    STR_TO_DATE(order_timestamp, '%Y-%m-%d'),
    '%Y-%m'
  ) AS order_month,
  ROUND(
    SUM(
      CASE
        WHEN order_status = 'Delivered'
          THEN quantity * unit_selling_price - discount_amount
        ELSE 0
      END
    ), 2
  ) AS delivered_gmv
FROM marketplace_transactions
GROUP BY order_month
ORDER BY order_month;
```

Step 3 — Monthly cancelled GMV

QUESTION 1 · STEP 3 — MONTHLY CANCELLED GMV

```sql
-- Monthly cancelled GMV
SELECT
  DATE_FORMAT(
    STR_TO_DATE(order_timestamp, '%Y-%m-%d'),
    '%Y-%m'
  ) AS order_month,
  ROUND(
    SUM(
      CASE
        WHEN order_status = 'Cancelled'
          THEN quantity * unit_selling_price - discount_amount
        ELSE 0
      END
    ), 2
  ) AS cancelled_gmv
FROM marketplace_transactions
GROUP BY order_month
ORDER BY order_month;
```

Step 4 — Monthly returned GMV

QUESTION 1 · STEP 4 — MONTHLY RETURNED GMV

```sql
-- Monthly returned GMV
SELECT
  DATE_FORMAT(
    STR_TO_DATE(order_timestamp, '%Y-%m-%d'),
    '%Y-%m'
  ) AS order_month,
  ROUND(
    SUM(
      CASE
        WHEN order_status = 'Returned'
          THEN quantity * unit_selling_price - discount_amount
        ELSE 0
      END
    ), 2
  ) AS returned_gmv
FROM marketplace_transactions
GROUP BY order_month
ORDER BY order_month;
```

Step 5 — Monthly discounts

QUESTION 1 · STEP 5 — MONTHLY DISCOUNTS

```sql
-- Monthly discounts
SELECT
  DATE_FORMAT(
    STR_TO_DATE(order_timestamp, '%Y-%m-%d'),
    '%Y-%m'
  ) AS order_month,
  ROUND(SUM(discount_amount), 2) AS total_discounts
FROM marketplace_transactions
GROUP BY order_month
ORDER BY order_month;
```

Step 6 — Monthly refunds

QUESTION 1 · STEP 6 — MONTHLY REFUNDS

```sql
-- Monthly refunds
SELECT
  DATE_FORMAT(
    STR_TO_DATE(order_timestamp, '%Y-%m-%d'),
    '%Y-%m'
  ) AS order_month,
  ROUND(SUM(refund_amount), 2) AS total_refunds
FROM marketplace_transactions
GROUP BY order_month
ORDER BY order_month;
```

Step 7 — Monthly seller commission

QUESTION 1 · STEP 7 — MONTHLY SELLER COMMISSION

```sql
-- Monthly seller commission
SELECT
  DATE_FORMAT(
    STR_TO_DATE(order_timestamp, '%Y-%m-%d'),
    '%Y-%m'
  ) AS order_month,
  ROUND(
    SUM(
      CASE
        WHEN order_status = 'Delivered'
          THEN (quantity * unit_selling_price - discount_amount)
               * seller_commission_rate
        ELSE 0
      END
    ), 2
  ) AS seller_commission
FROM marketplace_transactions
GROUP BY order_month
ORDER BY order_month;
```

Step 8 — Monthly payment fees

QUESTION 1 · STEP 8 — MONTHLY PAYMENT FEES

```sql
-- Monthly payment fees
SELECT
  DATE_FORMAT(
    STR_TO_DATE(order_timestamp, '%Y-%m-%d'),
    '%Y-%m'
  ) AS order_month,
  ROUND(
  SUM(
    CASE
      WHEN order_status = 'Delivered'
        THEN payment_fee
      ELSE 0
    END
  ), 2
  ) AS payment_fees
FROM marketplace_transactions
GROUP BY order_month
ORDER BY order_month;
```

Step 9 — Monthly fulfilment costs

QUESTION 1 · STEP 9 — MONTHLY FULFILMENT COSTS

```sql
-- Monthly fulfilment costs
SELECT
  DATE_FORMAT(
    STR_TO_DATE(order_timestamp, '%Y-%m-%d'),
    '%Y-%m'
  ) AS order_month,
  ROUND(
  SUM(
    CASE
      WHEN order_status = 'Delivered'
        THEN fulfilment_cost
      ELSE 0
    END
  ), 2
  ) AS fulfilment_costs
FROM marketplace_transactions
GROUP BY order_month
ORDER BY order_month;
```

Step 10 — Monthly contribution proxy

QUESTION 1 · STEP 10 — MONTHLY CONTRIBUTION PROXY

```sql
-- Monthly contribution proxy
SELECT
  DATE_FORMAT(
    STR_TO_DATE(order_timestamp, '%Y-%m-%d'),
    '%Y-%m'
  ) AS order_month,
  ROUND(
    SUM(
      CASE WHEN order_status = 'Delivered'
        THEN (quantity * unit_selling_price - discount_amount)
             - refund_amount
```

```
            - payment_fee
            - fulfilment_cost
        ELSE 0
      END
    ), 2
  ) AS contribution_proxy
FROM marketplace_transactions
GROUP BY order_month
ORDER BY order_month;
```

OUTPUT — STEPS 3, 4, 5, 7, 8 AND 9 (12 ROWS EACH, AS PRINTED BY EACH STEP)

order_month	cancelled_ gmv	returned_ gmv	total_ discounts	seller_ commission	payment_ fees	fulfilment_ costs
2025-01	5,288.67	1,457.70	10,025.80	5,678.34	1,232.67	3,922.59
2025-02	676.13	3,211.55	10,602.83	5,556.19	1,261.39	3,637.66
2025-03	6,686.20	4,438.13	14,404.60	6,608.44	1,399.52	4,364.81
2025-04	1,363.19	1,159.53	9,225.72	4,020.64	845.76	3,302.69
2025-05	1,080.45	4,051.34	8,730.23	5,184.40	1,232.94	3,724.26
2025-06	3,939.08	10,124.94	9,040.41	3,185.15	766.69	3,526.17
2025-07	2,029.46	3,345.50	11,002.45	5,120.16	1,408.38	4,385.90
2025-08	3,014.34	2,316.36	8,355.08	4,192.68	968.64	3,453.05
2025-09	4,267.44	3,732.00	10,341.69	4,844.16	1,165.88	4,098.84
2025-10	4,499.51	5,778.27	8,995.98	3,195.33	749.39	3,115.40
2025-11	1,314.99	5,671.41	10,637.18	3,164.16	899.32	3,000.67
2025-12	3,446.44	4,334.84	14,593.35	5,002.20	1,321.91	3,529.65

The outputs of Steps 1, 2, 6 and 10 were not printed on their own; they appear as columns of the combined bridge below.

Step 11 — Combined bridge with month-over-month growth

QUESTION 1 · STEP 11 — COMBINED BRIDGE WITH MONTH-OVER-MONTH GROWTH

```
-- Combined bridge with month-over-month growth
WITH monthly AS (
  SELECT
    DATE_FORMAT(
      STR_TO_DATE(order_timestamp, '%Y-%m-%d'),
      '%Y-%m'
    ) AS order_month,
    COUNT(DISTINCT order_id) AS order_count,
    SUM(quantity * unit_selling_price - discount_amount)
      AS total_gmv,
    SUM(CASE WHEN order_status = 'Delivered'
        THEN quantity * unit_selling_price - discount_amount
        ELSE 0 END) AS delivered_gmv,
    SUM(CASE WHEN order_status = 'Cancelled'
        THEN quantity * unit_selling_price - discount_amount
        ELSE 0 END) AS cancelled_gmv,
    SUM(CASE WHEN order_status = 'Returned'
        THEN quantity * unit_selling_price - discount_amount
        ELSE 0 END) AS returned_gmv,
    SUM(discount_amount) AS total_discounts,
    SUM(refund_amount) AS total_refunds,
    SUM(CASE WHEN order_status = 'Delivered'
        THEN (quantity * unit_selling_price - discount_amount)
             * seller_commission_rate
        ELSE 0 END) AS seller_commission,
    SUM(CASE WHEN order_status = 'Delivered'
        THEN payment_fee ELSE 0 END) AS payment_fees,
    SUM(CASE WHEN order_status = 'Delivered'
        THEN fulfilment_cost ELSE 0 END) AS fulfilment_costs,
    SUM(CASE WHEN order_status = 'Delivered'
        THEN (quantity * unit_selling_price - discount_amount)
             - refund_amount - payment_fee - fulfilment_cost
        ELSE 0 END) AS contribution_proxy
  FROM marketplace_transactions
  GROUP BY order_month
),
growth AS (
  SELECT
    *,
    ROUND(
      (order_count - LAG(order_count) OVER (ORDER BY order_month))
```

```
      * 100.0 /
      NULLIF(LAG(order_count) OVER (ORDER BY order_month), 0),
      2
    ) AS order_count_growth_pct,
    ROUND(
      (total_gmv - LAG(total_gmv) OVER (ORDER BY order_month))
      * 100.0 /
      NULLIF(LAG(total_gmv) OVER (ORDER BY order_month), 0),
      2
    ) AS gmv_growth_pct,
    ROUND(
      (delivered_gmv - LAG(delivered_gmv) OVER (ORDER BY order_month))
      * 100.0 /
      NULLIF(LAG(delivered_gmv) OVER (ORDER BY order_month), 0),
      2
    ) AS delivered_gmv_growth_pct,
    ROUND(
      (contribution_proxy -
       LAG(contribution_proxy) OVER (ORDER BY order_month))
      * 100.0 /
      NULLIF(LAG(contribution_proxy) OVER (ORDER BY order_month), 0),
      2
    ) AS contribution_growth_pct
  FROM monthly
)
SELECT
  order_month,
  order_count,
  ROUND(total_gmv, 2) AS total_gmv,
  ROUND(delivered_gmv, 2) AS delivered_gmv,
  ROUND(cancelled_gmv, 2) AS cancelled_gmv,
  ROUND(returned_gmv, 2) AS returned_gmv,
  ROUND(total_discounts, 2) AS total_discounts,
  ROUND(total_refunds, 2) AS total_refunds,
  ROUND(seller_commission, 2) AS seller_commission,
  ROUND(payment_fees, 2) AS payment_fees,
  ROUND(fulfilment_costs, 2) AS fulfilment_costs,
  ROUND(contribution_proxy, 2) AS contribution_proxy,
  order_count_growth_pct,
  gmv_growth_pct,
  delivered_gmv_growth_pct,
  contribution_growth_pct
FROM growth
ORDER BY order_month;
```

OUTPUT — 12 ROWS (PART A: ORDERS AND GMV BY STATUS)

order_month	order_count	total_gmv	delivered_gmv	cancelled_gmv	returned_gmv
2025-01	74	50,340.60	41,622.30	5,288.67	1,457.70
2025-02	71	46,456.72	42,502.85	676.13	3,211.55
2025-03	81	61,712.60	49,186.15	6,686.20	4,438.13
2025-04	61	37,012.57	32,005.37	1,363.19	1,159.53
2025-05	71	47,467.78	41,244.88	1,080.45	4,051.34
2025-06	69	42,177.12	27,704.86	3,939.08	10,124.94
2025-07	78	54,038.29	47,015.45	2,029.46	3,345.50
2025-08	65	40,984.74	34,999.66	3,014.34	2,316.36
2025-09	81	47,570.73	38,529.06	4,267.44	3,732.00
2025-10	68	42,816.36	24,875.36	4,499.51	5,778.27
2025-11	65	36,213.83	27,903.66	1,314.99	5,671.41
2025-12	77	54,526.66	40,317.50	3,446.44	4,334.84

OUTPUT — 12 ROWS (PART B: DISCOUNTS, COSTS AND CONTRIBUTION)

order_month	total_discounts	total_refunds	seller_commission	payment_fees	fulfilment_costs	contribution_proxy
2025-01	10,025.80	1,412.26	5,678.34	1,232.67	3,922.59	36,467.04
2025-02	10,602.83	2,865.51	5,556.19	1,261.39	3,637.66	37,603.80
2025-03	14,404.60	2,882.47	6,608.44	1,399.52	4,364.81	43,421.82
2025-04	9,225.72	1,226.17	4,020.64	845.76	3,302.69	27,856.92

order_month	total_ discounts	total_ refunds	seller_ commission	payment_ fees	fulfilment_ costs	contribution_ proxy
2025-05	8,730.23	2,709.53	5,184.40	1,232.94	3,724.26	36,287.68
2025-06	9,040.41	8,236.48	3,185.15	766.69	3,526.17	23,412.00
2025-07	11,002.45	2,623.37	5,120.16	1,408.38	4,385.90	41,221.17
2025-08	8,355.08	1,260.60	4,192.68	968.64	3,453.05	30,577.97
2025-09	10,341.69	3,390.61	4,844.16	1,165.88	4,098.84	33,264.34
2025-10	8,995.98	4,780.92	3,195.33	749.39	3,115.40	21,010.57
2025-11	10,637.18	3,868.39	3,164.16	899.32	3,000.67	24,003.67
2025-12	14,593.35	2,688.98	5,002.20	1,321.91	3,529.65	35,465.94

OUTPUT — 12 ROWS (PART C: MONTH-OVER-MONTH GROWTH, % · SHADED = GMV AND CONTRIBUTION MOVE IN OPPOSITE DIRECTIONS)

order_month	order_count_ growth_pct	gmv_growth_ pct	delivered_ gmv_growth_ pct	contribution_ growth_pct
2025-01	NULL	NULL	NULL	NULL
2025-02	-4.05	-7.72	2.12	3.12
2025-03	14.08	32.84	15.72	15.47
2025-04	-24.69	-40.02	-34.93	-35.85
2025-05	16.39	28.25	28.87	30.26
2025-06	-2.82	-11.15	-32.83	-35.48
2025-07	13.04	28.12	69.70	76.07
2025-08	-16.67	-24.16	-25.56	-25.82
2025-09	24.62	16.07	10.08	8.79
2025-10	-16.05	-9.99	-35.44	-36.84
2025-11	-4.41	-15.42	12.17	14.25
2025-12	18.46	50.57	44.49	47.75

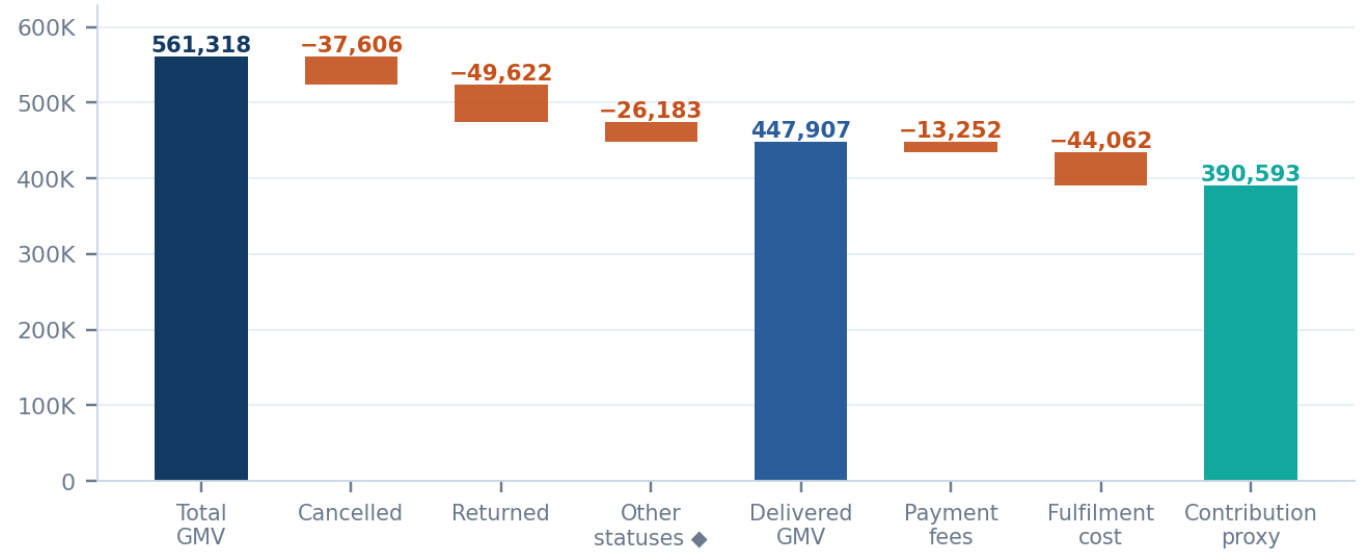


Figure 4.1 — Full-year bridge from total GMV to contribution proxy ◆. "Other statuses" is the residual that total GMV leaves after delivered, cancelled and returned GMV.

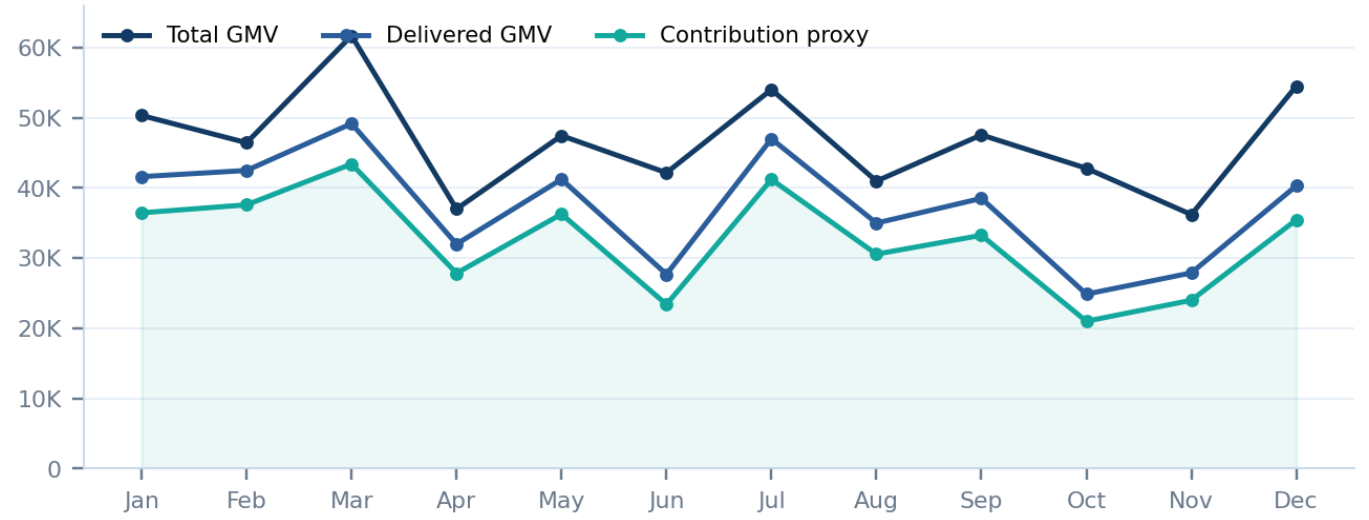


Figure 4.2 — Monthly total GMV, delivered GMV and contribution proxy.

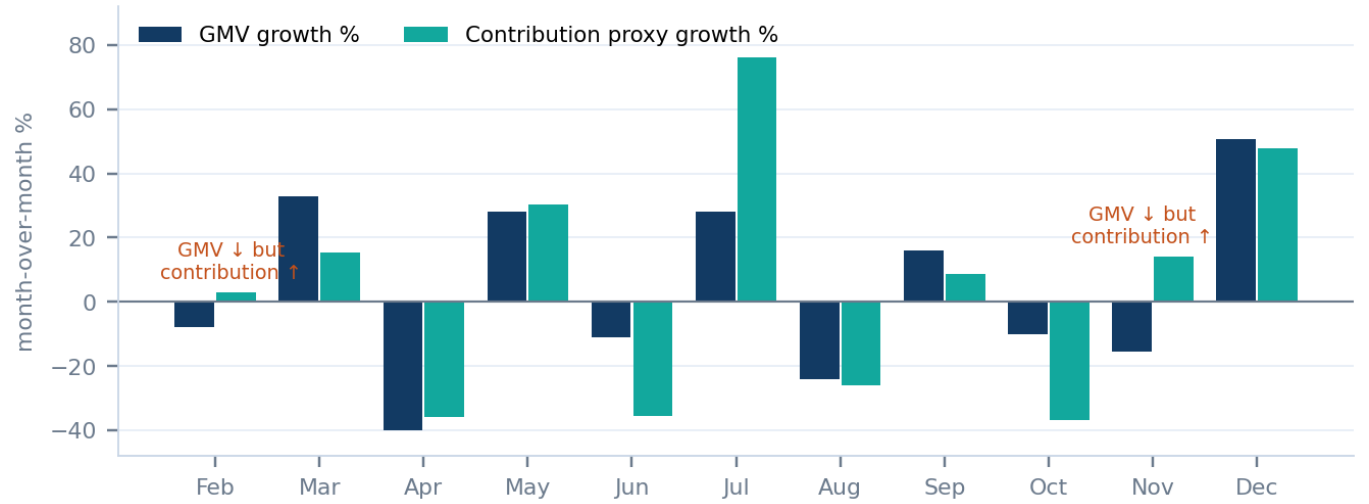


Figure 4.3 — Month-over-month growth in GMV and in contribution proxy.

KEY INSIGHT

No month has positive GMV growth with negative contribution growth. The five months in which GMV rose (March, May, July, September, December) all show contribution rising too, by 15.47%, 30.26%, 76.07%, 8.79% and 47.75%. The opposite pattern appears twice: **February** (GMV −7.72%, contribution +3.12%) and **November** (GMV −15.42%, contribution +14.25%). The reason is structural: the contribution proxy is built on delivered rows only, so it tracks delivered GMV, never differing from its growth by more than 6.37 points. Total GMV also contains cancelled, returned and open orders, which is why June's total GMV fell only 11.15% while delivered GMV fell 32.83% — returned GMV jumped to 10,124.94 and refunds to 8,236.48.

KEY INSIGHT

Over the year total GMV of 561,318.00 ◆ resolves to delivered GMV of 447,907.10 (79.8%), cancelled 37,605.90 (6.7%), returned 49,621.57 (8.8%) and 26,183.43 (4.7%) in other statuses. Payment fees take 2.96% of delivered GMV and fulfilment 9.84%, leaving a contribution proxy of 390,592.92 (87.2%). It peaks in March (43,421.82) and bottoms out in October (21,010.57). Seller commission (55,751.85) is calculated in the bridge but not used in the proxy. ◆ Derived figures are simple arithmetic on the printed outputs (for example summing twelve monthly values); they were not produced by a query.

Question 2 — Customer concentration and high-value, negative-contribution customers

Calculate the share of delivered GMV and contribution generated by the top 1%, 5% and 10% of customers, assess repeat-purchase participation, identify customers above the 90th GMV percentile with negative contribution, and determine whether the largest customers are also the most profitable.

Step 1 — Delivered GMV and contribution by customer

QUESTION 2 · STEP 1 — DELIVERED GMV AND CONTRIBUTION BY CUSTOMER

```sql
-- Step 1: delivered GMV and contribution proxy by customer
USE case_study_1;

SELECT
  customer_id,
  ROUND(
    SUM(
      quantity * unit_selling_price - discount_amount
    ), 2
  ) AS delivered_gmv,
  ROUND(
    SUM(
      (quantity * unit_selling_price - discount_amount)
      - refund_amount
      - payment_fee
      - fulfilment_cost
    ), 2
  ) AS contribution_proxy,
  COUNT(DISTINCT order_id) AS delivered_order_count
FROM marketplace_transactions
WHERE order_status = 'Delivered'
GROUP BY customer_id
ORDER BY delivered_gmv DESC;
```

OUTPUT — TOP 29 OF 226 CUSTOMERS (FIRST 29 ROWS PRINTED IN THE SOURCE)

customer_id	delivered_gmv	contribution_proxy	delivered_order_count
C0100	8,974.79	8,461.59	3
C0097	6,900.56	6,245.43	7
C0002	6,831.47	6,415.45	4
C0131	6,263.04	5,753.40	5
C0010	6,044.40	5,515.53	4
C0239	5,890.59	5,428.95	5
C0129	5,866.50	5,336.17	7
C0150	5,827.17	5,360.28	5
C0189	5,790.08	5,211.15	8
C0085	5,624.01	5,218.64	3
C0024	5,440.20	5,038.57	3
C0165	5,429.14	4,995.81	5
C0064	5,110.32	4,517.60	7
C0114	5,090.27	4,821.65	3
C0147	4,940.15	4,264.27	8
C0109	4,853.37	4,574.81	4
C0075	4,735.04	4,327.85	5
C0091	4,720.18	4,405.80	2
C0134	4,630.82	4,248.31	3
C0027	4,622.33	4,278.21	4
C0170	4,398.26	3,731.31	8
C0045	4,244.39	3,978.85	3

customer_id	delivered_gmv	contribution_proxy	delivered_order_count
C0151	4,158.12	3,507.53	8
C0233	4,059.17	3,768.63	3
C0021	4,027.33	3,668.99	6
C0220	3,903.71	3,497.09	4
C0040	3,702.39	3,264.04	5
C0127	3,614.54	3,061.55	7
C0048	3,546.90	3,138.64	4

Step 2 — Customer concentration

QUESTION 2 · STEP 2 — CUSTOMER CONCENTRATION

```sql
-- Step 2: customer concentration
USE case_study_1;

WITH customer_totals AS (
  SELECT
    customer_id,
    SUM(quantity * unit_selling_price - discount_amount) AS delivered_gmv,
    SUM(
      (quantity * unit_selling_price - discount_amount)
      - refund_amount
      - payment_fee
      - fulfilment_cost
    ) AS contribution_proxy
  FROM marketplace_transactions
  WHERE order_status = 'Delivered'
  GROUP BY customer_id
),
ranked_customers AS (
  SELECT
    customer_id,
    delivered_gmv,
    contribution_proxy,
    NTILE(100) OVER (
      ORDER BY delivered_gmv DESC
    ) AS pct_bucket
  FROM customer_totals
),
concentration AS (
  SELECT
    'Top 1%' AS concentration_band,
    1 AS sort_order,
    SUM(delivered_gmv) AS delivered_gmv,
    SUM(contribution_proxy) AS contribution_proxy
  FROM ranked_customers
  WHERE pct_bucket <= 1
  UNION ALL
  SELECT
    'Top 5%',
    2,
    SUM(delivered_gmv),
    SUM(contribution_proxy)
  FROM ranked_customers
  WHERE pct_bucket <= 5
  UNION ALL
  SELECT
    'Top 10%',
    3,
    SUM(delivered_gmv),
    SUM(contribution_proxy)
  FROM ranked_customers
  WHERE pct_bucket <= 10
)
SELECT
  concentration_band,
  ROUND(delivered_gmv, 2) AS delivered_gmv,
  ROUND(contribution_proxy, 2) AS contribution_proxy,
  ROUND(
    delivered_gmv * 100 /
    (SELECT SUM(delivered_gmv) FROM ranked_customers),
    2
  ) AS share_of_total_gmv_pct,
  ROUND(
    contribution_proxy * 100 /
    (SELECT SUM(contribution_proxy) FROM ranked_customers),
```

```
    2
  ) AS share_of_total_contribution_pct
FROM concentration
ORDER BY sort_order;
```

OUTPUT — 3 ROWS

concentration_band	delivered_gmv	contribution_proxy	share_of_total_gmv_pct	share_of_total_contribution_pct
Top 1%	22,706.82	21,122.47	5.07	5.41
Top 5%	90,022.69	82,584.49	20.10	21.14
Top 10%	152,736.03	139,119.76	34.10	35.62

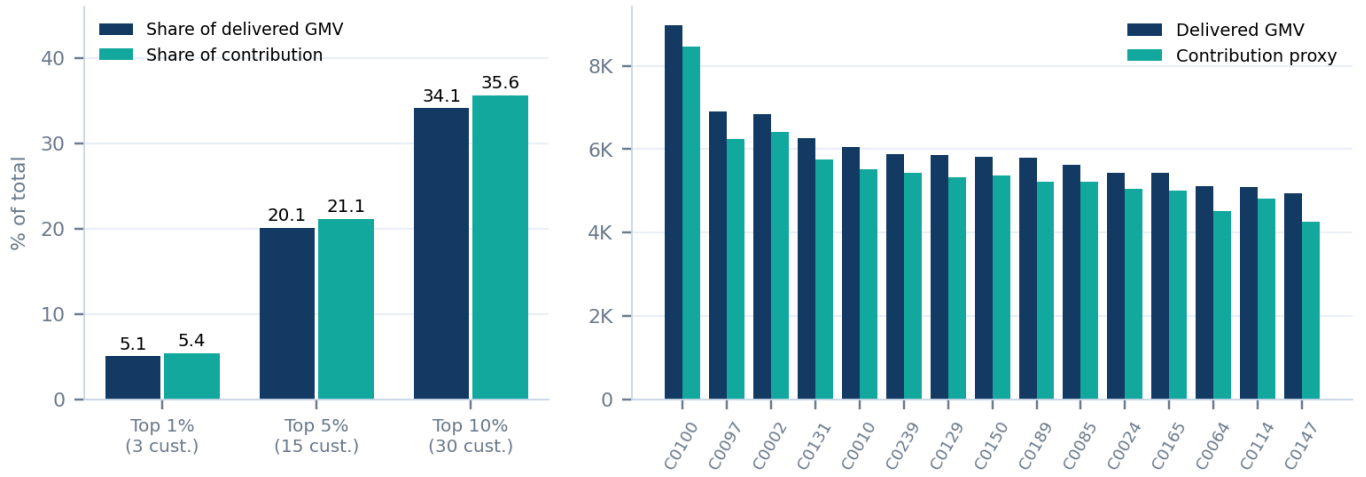


Figure 4.4 — Concentration bands (left) and the fifteen largest customers by delivered GMV with their contribution (right). Customer counts in the bands follow from how NTILE(100) divides 226 customers.

Step 3 — High-value customers with negative contribution

QUESTION 2 · STEP 3 — HIGH-VALUE BUT UNPROFITABLE CUSTOMERS

```
-- Step 3: high-value but unprofitable customers
USE case_study_1;

WITH customer_totals AS (
  SELECT
    customer_id,
    SUM(quantity * unit_selling_price - discount_amount) AS delivered_gmv,
    SUM(
      (quantity * unit_selling_price - discount_amount)
      - refund_amount
      - payment_fee
      - fulfilment_cost
    ) AS contribution_proxy,
    COUNT(DISTINCT order_id) AS delivered_order_count
  FROM marketplace_transactions
  WHERE order_status = 'Delivered'
  GROUP BY customer_id
),
ranked_customers AS (
  SELECT
    customer_id,
    delivered_gmv,
    contribution_proxy,
    delivered_order_count,
    PERCENT_RANK() OVER (
      ORDER BY delivered_gmv
    ) AS gmv_percentile
  FROM customer_totals
)
SELECT
  customer_id,
  ROUND(delivered_gmv, 2) AS delivered_gmv,
  ROUND(contribution_proxy, 2) AS contribution_proxy,
  delivered_order_count,
  ROUND(gmv_percentile, 2) AS gmv_percentile,
  CASE
    WHEN contribution_proxy < 0 THEN 'Yes'
    ELSE 'No'
  END AS negative_contribution
```

```sql
FROM ranked_customers
WHERE contribution_proxy < 0
ORDER BY delivered_gmv DESC;
```

OUTPUT — 0 ROWS (EMPTY RESULT SET; COLUMNS: CUSTOMER_ID, DELIVERED_GMV, CONTRIBUTION_PROXY, DELIVERED_ORDER_COUNT, GMV_PERCENTILE, NEGATIVE_CONTRIBUTION)

Step 4 — Count negative-contribution customers

QUESTION 2 · STEP 4 — CHECK NEGATIVE-CONTRIBUTION CUSTOMERS

```sql
-- Step 4: check negative-contribution customers
WITH customer_totals AS (
  SELECT
    customer_id,
    SUM(
      (quantity * unit_selling_price - discount_amount)
      - refund_amount
      - payment_fee
      - fulfilment_cost
    ) AS contribution_proxy
  FROM marketplace_transactions
  WHERE order_status = 'Delivered'
  GROUP BY customer_id
)
SELECT
  COUNT(*) AS total_customers,
  SUM(CASE
    WHEN contribution_proxy < 0 THEN 1
    ELSE 0
  END) AS negative_contribution_customers
FROM customer_totals;
```

OUTPUT — 1 ROW

total_customers	negative_contribution_customers
226	0

KEY INSIGHT

Yes — in this data the largest customers are also the most profitable. In every band the share of contribution exceeds the share of GMV (5.41% vs 5.07%, 21.14% vs 20.10%, 35.62% vs 34.10%), and the three largest customers convert 93.0% of delivered GMV into contribution against 87.2% for the whole base ◆. The number-one customer on GMV (C0100, 8,974.79) is also number one on contribution (8,461.59); only the second and third swap, with C0002 (6,415.45) ahead of C0097 (6,245.43). None of the 226 customers has negative contribution, so no high-GMV customer is hiding a loss. Concentration is moderate: the top 30 customers hold 34.10% of delivered GMV and the largest customer is 4.5× the average of 1,981.89 ◆. Every one of the 29 customers listed has at least two delivered orders; overall repeat participation is reported by segment in Question 3, Step 7. **Two caveats:** the “top 1/5/10%” bands are really the top 3, 15 and 30 customers (Defect 3), and the proxy ignores product cost, so a margin near 90% is a GMV-based ceiling, not a profit margin.

SECTION 05

Discount, Cohort & Scenario Analytics

Questions 3–5: how much of the business is bought with discount, how acquisition cohorts behave, and how far volume can fall before a discount cut stops paying.

DISCOUNT-RATE RANGE	DEEPEST CHANNEL	REPEAT RATE, STANDARD	TOP COHORT CONTRIBUTION	TIGHTEST BREAK-EVEN
13.2–16.5%	17.06%	66.67%	70,938	2.09%
by category · Sports to Beauty	Affiliate · Social is 14.04%	High Value: 11.67%	2023-01 · 32 customers	Sports · 10% discount cut

Question 3 — Discount dependency and commercial quality

Calculate discount rates by category and acquisition channel, find categories where higher discounting coincides with lower contribution per delivered order, compare customer segments on average order value, repeat-purchase frequency, discount dependency, refund burden and contribution, and identify high-value transactions with weak post-discount economics. Steps 8–9 add the contribution exposed to a 10% volume reduction.

Step 1 — Discount rate by product category

QUESTION 3 · STEP 1 — DISCOUNT RATE BY PRODUCT CATEGORY

```sql
-- Step 1: discount rate by product category
USE case_study_1;

SELECT
  product_category,
  ROUND(
    100 * SUM(discount_amount) /
    NULLIF(SUM(quantity * unit_list_price), 0),
    2
  ) AS discount_rate_pct
FROM marketplace_transactions
GROUP BY product_category
ORDER BY discount_rate_pct DESC;
```

OUTPUT — 5 ROWS

product_category	discount_rate_pct
Beauty	16.53
Fashion	16.46
Home & Kitchen	16.39
Electronics	14.49
Sports	13.22

Step 2 — Discount rate by acquisition channel

QUESTION 3 · STEP 2 — DISCOUNT RATE BY ACQUISITION CHANNEL

```sql
-- Step 2: discount rate by acquisition channel
SELECT
  acquisition_channel,
  ROUND(
    100 * SUM(discount_amount) /
    NULLIF(SUM(quantity * unit_list_price), 0),
    2
  ) AS discount_rate_pct
FROM marketplace_transactions
GROUP BY acquisition_channel
ORDER BY discount_rate_pct DESC;
```

OUTPUT — 5 ROWS

acquisition_channel	discount_rate_pct
Affiliate	17.06

acquisition_channel	discount_rate_pct
Referral	15.91
Organic	15.35
Paid Search	14.97
Social	14.04

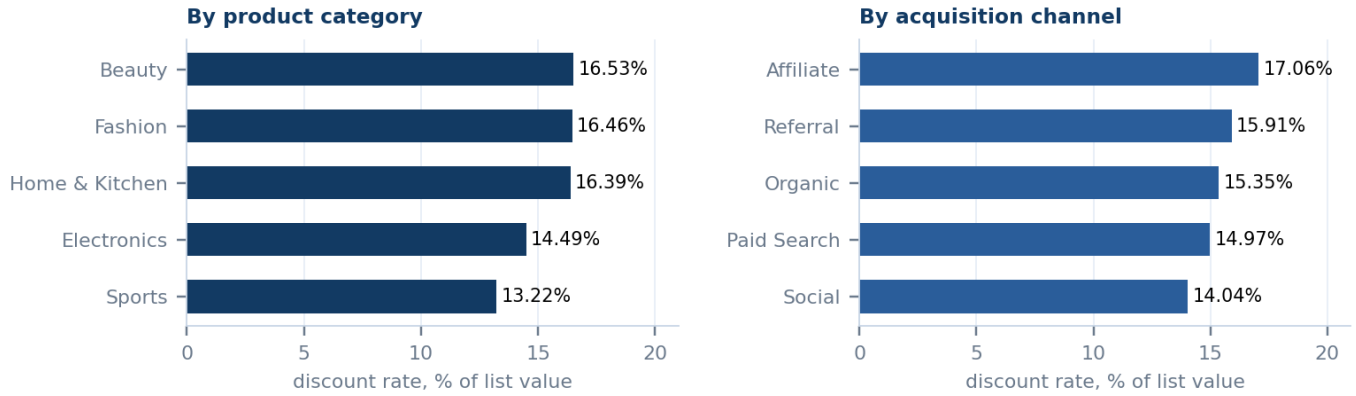


Figure 5.1 — Discount rate (discount ÷ list value, all order statuses) by category and by acquisition channel.

Step 3 — Category-wise commercial quality

QUESTION 3 · STEP 3 — CATEGORY-WISE COMMERCIAL QUALITY

```sql
-- Step 3: category-wise commercial quality
WITH category_metrics AS (
  SELECT
    product_category,
    100 * SUM(discount_amount) /
      NULLIF(SUM(quantity * unit_list_price), 0)
      AS discount_rate_pct,
    SUM(
      CASE WHEN order_status = 'Delivered'
      THEN (quantity * unit_selling_price - discount_amount)
           - refund_amount - payment_fee - fulfilment_cost
      ELSE 0 END
    ) / NULLIF(
      COUNT(DISTINCT CASE
        WHEN order_status = 'Delivered' THEN order_id
      END), 0
    ) AS contribution_per_delivered_order,
    100 * SUM(
      CASE WHEN order_status = 'Delivered'
      THEN refund_amount ELSE 0 END
    ) / NULLIF(
      SUM(CASE
        WHEN order_status = 'Delivered'
        THEN quantity * unit_selling_price - discount_amount
        ELSE 0
      END), 0
    ) AS refund_rate_pct
  FROM marketplace_transactions
  GROUP BY product_category
)
SELECT
  product_category,
  ROUND(discount_rate_pct, 2) AS discount_rate_pct,
  ROUND(contribution_per_delivered_order, 2)
    AS contribution_per_delivered_order,
  ROUND(refund_rate_pct, 2) AS refund_rate_pct,
  CASE
    WHEN discount_rate_pct >= 16
     AND refund_rate_pct >= 5 THEN 'High'
    WHEN discount_rate_pct >= 15
      OR refund_rate_pct >= 4 THEN 'Medium'
    ELSE 'Low'
  END AS commercial_risk
FROM category_metrics
ORDER BY contribution_per_delivered_order ASC;
```

OUTPUT — 5 ROWS

product_category	discount_rate_pct	contribution_per_delivered_order	refund_rate_pct	commercial_risk
Fashion	16.46	504.34	0	Medium
Sports	13.22	546.04	0	Low
Home & Kitchen	16.39	603.34	0	Medium
Beauty	16.53	618.88	0	Medium
Electronics	14.49	721.55	0	Low

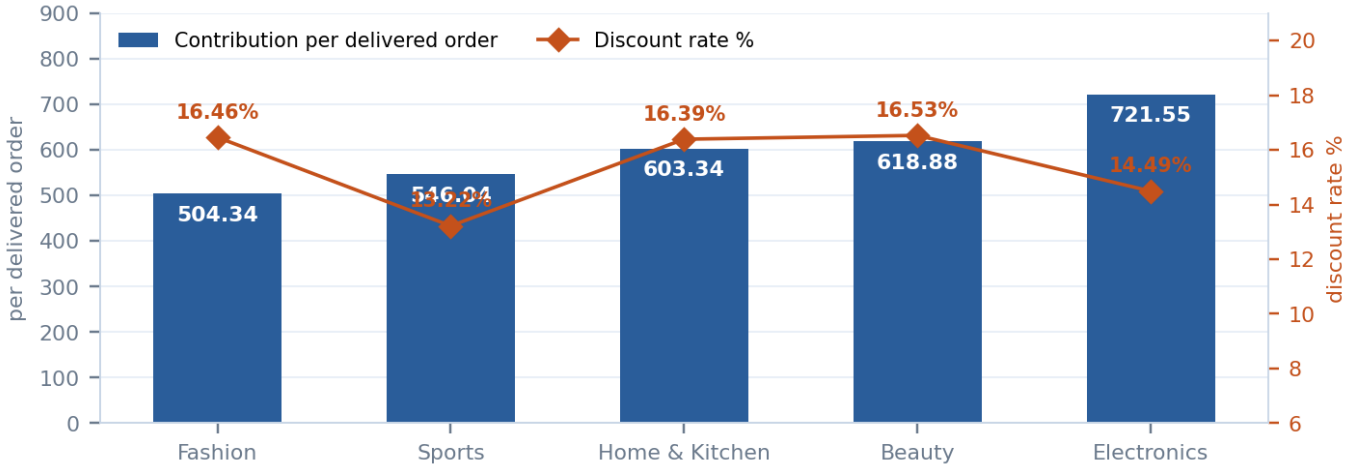


Figure 5.2 — Contribution per delivered order against discount rate, by category.

KEY INSIGHT

Discount rate alone does not explain contribution per order. Fashion pairs a high discount (16.46%) with the lowest contribution per delivered order (504.34), but Sports has the lowest discount (13.22%) and the second-lowest contribution (546.04), and Beauty, with the highest discount (16.53%), ranks fourth of five (618.88). Electronics leads on contribution (721.55) with a below-average discount of 14.49%. Refund rate prints as 0 for every category, so the "High" risk rule (discount ≥ 16% and refund ≥ 5%) can never fire; Fashion, Home & Kitchen and Beauty are labelled Medium on discount alone (Defect 2).

Step 4 — High-value transactions with weak post-discount economics

QUESTION 3 · STEP 4 — HIGH-VALUE TRANSACTIONS WITH WEAK POST-DISCOUNT ECONOMICS

```sql
-- Step 4: high-value transactions with weak post-discount economics
SELECT
  transaction_id,
  order_id,
  customer_id,
  product_category,
  quantity,
  ROUND(quantity * unit_list_price, 2) AS list_price_value,
  ROUND(discount_amount, 2) AS discount_amount,
  ROUND(
    quantity * unit_selling_price - discount_amount,
    2
  ) AS delivered_gmv,
  ROUND(
    (quantity * unit_selling_price - discount_amount)
    - refund_amount
    - payment_fee
    - fulfilment_cost,
    2
  ) AS contribution_proxy
FROM marketplace_transactions
WHERE order_status = 'Delivered'
ORDER BY contribution_proxy ASC
LIMIT 10;
```

OUTPUT — 10 ROWS

transaction_id	order_id	customer_id	product_category	quantity	list_price_value	discount_amount	delivered_gmv	contribution_proxy
T00354	O00211	C0023	Sports	1	371.96	167.38	37.20	-72.44
T00268	O00362	C0001	Fashion	1	221.97	96.45	29.07	-62.22
T00348	O00682	C0003	Fashion	1	182.75	65.18	52.39	-59.30
T00647	O00087	C0224	Home & Kitchen	1	125.73	37.51	50.71	-57.66
T00894	O00332	C0124	Electronics	1	455.47	204.96	45.55	-55.04
T00113	O00061	C0053	Beauty	1	594.03	267.31	59.41	-51.14
T00324	O00064	C0170	Electronics	1	274.63	123.58	27.47	-42.57
T00248	O00185	C0004	Home & Kitchen	2	488.88	206.72	75.44	-39.23
T00509	O00712	C0206	Sports	1	331.08	148.99	33.10	-37.12
T00344	O00606	C0138	Electronics	1	133.34	30.76	71.82	-28.01

Step 5 — High-discount, low-contribution transactions

QUESTION 3 · STEP 5 — HIGH-DISCOUNT, LOW-CONTRIBUTION TRANSACTIONS

```sql
-- Step 5: high-discount, low-contribution transactions
SELECT
  transaction_id,
  order_id,
  customer_id,
  product_category,
  ROUND(
    100 * discount_amount /
    NULLIF(quantity * unit_list_price, 0), 2
  ) AS discount_rate_pct,
  ROUND(
    quantity * unit_selling_price - discount_amount, 2
  ) AS delivered_gmv,
  ROUND(
    (quantity * unit_selling_price - discount_amount)
    - refund_amount - payment_fee - fulfilment_cost, 2
  ) AS contribution_proxy
FROM marketplace_transactions
WHERE order_status = 'Delivered'
ORDER BY discount_rate_pct DESC, contribution_proxy ASC
LIMIT 10;
```

OUTPUT — 10 ROWS

transaction_id	order_id	customer_id	product_category	discount_rate_pct	delivered_gmv	contribution_proxy
T00354	O00211	C0023	Sports	45	37.20	-72.44
T00894	O00332	C0124	Electronics	45	45.55	-55.04
T00113	O00061	C0053	Beauty	45	59.41	-51.14
T00324	O00064	C0170	Electronics	45	27.47	-42.57
T00509	O00712	C0206	Sports	45	33.10	-37.12
T00642	O00369	C0145	Home & Kitchen	45	65.75	-26.79
T00018	O00283	C0221	Home & Kitchen	45	47.22	-13.63
T00745	O00080	C0193	Fashion	45	49.23	-7.11
T00286	O00726	C0116	Sports	45	20.16	0.74
T00760	O00542	C0024	Home & Kitchen	45	86.41	43.08

KEY INSIGHT

The ten highest-discount delivered lines all sit at exactly **45%** of list value, which looks like a discount cap. Eight of the ten have negative contribution. These rows should be read with care: the delivered GMV shown (for example 37.20 on a line with a list value of 371.96) is the list value minus the discount *twice*, and once the double deduction is removed nine of the ten weakest lines in Step 4 turn positive (Figure 6.1, Defect 1). Step 4 also does not filter on value, so it lists low-value lines rather than high-value ones (Defect 7).

Step 6 — Customer segment analysis

QUESTION 3 · STEP 6 — CUSTOMER SEGMENT ANALYSIS

```sql
-- Step 6: customer segment analysis
SELECT
  customer_segment,
  COUNT(DISTINCT customer_id) AS total_customers,
  COUNT(DISTINCT CASE
    WHEN order_status = 'Delivered' THEN order_id
  END) AS delivered_orders,

  ROUND(
    SUM(CASE WHEN order_status = 'Delivered'
      THEN quantity * unit_selling_price - discount_amount
      ELSE 0 END)
    / NULLIF(COUNT(DISTINCT CASE
      WHEN order_status = 'Delivered' THEN order_id
    END), 0), 2
  ) AS avg_order_value,

  ROUND(
    100 * SUM(CASE WHEN order_status = 'Delivered'
      THEN discount_amount ELSE 0 END)
    / NULLIF(SUM(CASE WHEN order_status = 'Delivered'
      THEN quantity * unit_list_price ELSE 0 END), 0), 2
  ) AS discount_rate_pct,

  ROUND(
    100 * SUM(CASE WHEN order_status = 'Delivered'
      THEN refund_amount ELSE 0 END)
    / NULLIF(SUM(CASE WHEN order_status = 'Delivered'
      THEN quantity * unit_selling_price - discount_amount
      ELSE 0 END), 0), 2
  ) AS refund_rate_pct,

  ROUND(
    SUM(CASE WHEN order_status = 'Delivered'
      THEN (quantity * unit_selling_price - discount_amount)
           - refund_amount - payment_fee - fulfilment_cost
      ELSE 0 END)
    / NULLIF(COUNT(DISTINCT CASE
      WHEN order_status = 'Delivered' THEN order_id
    END), 0), 2
  ) AS contribution_per_order
FROM marketplace_transactions
GROUP BY customer_segment
ORDER BY contribution_per_order DESC;
```

OUTPUT — 3 ROWS

customer_segment	total_customers	delivered_orders	avg_order_value	discount_rate_pct	refund_rate_pct	contribution_per_order
Standard	216	333	894.33	15.26	0	784.99
Premium	146	175	654.24	16.51	0	564.83
High Value	73	67	531.40	15.91	0	452.90

Step 7 — Repeat-purchase frequency

QUESTION 3 · STEP 7 — REPEAT-PURCHASE FREQUENCY

```sql
-- Step 7: repeat-purchase frequency
WITH customer_orders AS (
  SELECT
    customer_id,
    customer_segment,
    COUNT(DISTINCT order_id) AS delivered_order_count
  FROM marketplace_transactions
  WHERE order_status = 'Delivered'
  GROUP BY customer_id, customer_segment
)
SELECT
  customer_segment,
  COUNT(*) AS purchasing_customers,
  SUM(CASE
    WHEN delivered_order_count >= 2 THEN 1
    ELSE 0
  END) AS repeat_customers,
  ROUND(
    100.0 * SUM(CASE
      WHEN delivered_order_count >= 2 THEN 1
      ELSE 0
    END) / COUNT(*), 2
```

```sql
  ) AS repeat_purchase_rate_pct,
  ROUND(AVG(delivered_order_count), 2) AS avg_delivered_orders_per_customer
FROM customer_orders
GROUP BY customer_segment
ORDER BY repeat_purchase_rate_pct DESC;
```

OUTPUT — 3 ROWS

customer_segment	purchasing_customers	repeat_customers	repeat_purchase_rate_pct	avg_delivered_orders_per_customer
Standard	204	136	66.67	2.24
Premium	130	47	36.15	1.52
High Value	60	7	11.67	1.13

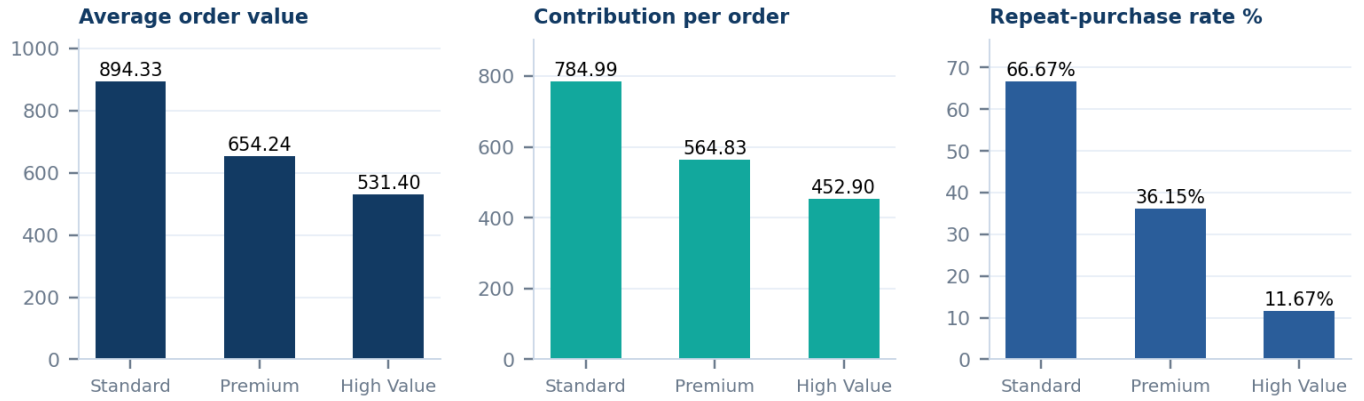


Figure 5.3 — Customer segments compared on order value, contribution per order and repeat-purchase rate.

KEY INSIGHT

The “High Value” segment is the weakest on every measure. It has the lowest average order value (531.40 vs 894.33 for Standard), the lowest contribution per order (452.90 vs 784.99) and by far the lowest repeat rate (11.67% vs 66.67%, with only 1.13 delivered orders per customer). Premium takes the deepest discount (16.51% against 15.26% for Standard). The segment labels therefore do not describe customer value, and customers are counted in more than one segment (394 customer-segment pairs against 226 customers, Defect 5). Refund rate is 0.00% in all segments for the reason given in Defect 2.

Step 8 — Category contribution ranking and the 10% GMV scenario

QUESTION 3 · STEP 8 — CATEGORY CONTRIBUTION RANKING

```sql
-- Step 8: category contribution ranking and 10% GMV scenario
SELECT
  product_category,

  ROUND(SUM(
    CASE WHEN order_status = 'Delivered'
    THEN quantity * unit_selling_price - discount_amount
    ELSE 0 END
  ), 2) AS delivered_gmv,

  ROUND(SUM(
    CASE WHEN order_status = 'Delivered'
    THEN (quantity * unit_selling_price - discount_amount)
         - refund_amount - payment_fee - fulfilment_cost
    ELSE 0 END
  ), 2) AS current_contribution_proxy,

  ROUND(
    10 * SUM(
      CASE WHEN order_status = 'Delivered'
      THEN quantity * unit_selling_price - discount_amount
      ELSE 0 END
    ) / 100, 2
  ) AS gmv_reduction_scenario_10pct
FROM marketplace_transactions
GROUP BY product_category
ORDER BY gmv_reduction_scenario_10pct DESC;
```

OUTPUT — 5 ROWS (RECONSTRUCTED ◆ FROM STEP 9, WHICH USES THE SAME EXPRESSIONS; THE CAPTURE PASTED UNDER STEP 8 IN THE SOURCE IS STEP 7'S TABLE)

product_ category	delivered_gmv	current_ contribution_ proxy	gmv_reduction_ scenario_10pct
Electronics	108,362.46	95,966.49	10,836.25
Beauty	101,829.78	89,737.69	10,182.98
Home & Kitchen	91,523.91	79,640.35	9,152.39
Fashion	74,792.31	63,546.35	7,479.23
Sports	71,398.64	61,702.04	7,139.86

Step 9 — Contribution proxy potentially exposed

QUESTION 3 · STEP 9 — CONTRIBUTION PROXY POTENTIALLY EXPOSED

```sql
-- Step 9: contribution proxy potentially exposed
WITH category_metrics AS (
  SELECT
    product_category,
    SUM(
      CASE WHEN order_status = 'Delivered'
      THEN quantity * unit_selling_price - discount_amount
      ELSE 0 END
    ) AS delivered_gmv,
    SUM(
      CASE WHEN order_status = 'Delivered'
      THEN (quantity * unit_selling_price - discount_amount)
           - refund_amount - payment_fee - fulfilment_cost
      ELSE 0 END
    ) AS contribution_proxy
  FROM marketplace_transactions
  GROUP BY product_category
)
SELECT
  product_category,
  ROUND(delivered_gmv, 2) AS delivered_gmv,
  ROUND(contribution_proxy, 2) AS current_contribution_proxy,
  ROUND(delivered_gmv * 0.10, 2) AS gmv_reduction_10pct,
  ROUND(contribution_proxy * 0.10, 2)
    AS contribution_proxy_potentially_exposed
FROM category_metrics
ORDER BY contribution_proxy_potentially_exposed DESC;
```

OUTPUT — 5 ROWS

product_ category	delivered_gmv	current_ contribution_ proxy	gmv_reduction_ 10pct	contribution_ proxy_ potentially_ exposed
Electronics	108,362.46	95,966.49	10,836.25	9,596.65
Beauty	101,829.78	89,737.69	10,182.98	8,973.77
Home & Kitchen	91,523.91	79,640.35	9,152.39	7,964.04
Fashion	74,792.31	63,546.35	7,479.23	6,354.64
Sports	71,398.64	61,702.04	7,139.86	6,170.20

KEY INSIGHT

A 10% fall in delivered volume would put 9,596.65 of contribution at risk in Electronics, the largest exposure, down to 6,170.20 in Sports. Electronics and Beauty together carry 47.5% of the contribution proxy ◆, so volume protection matters most there even though discount sensitivity (Question 5) is highest in Sports.

Question 4 — Acquisition cohort quality

The question asks for acquisition-month cohorts compared on first-purchase contribution, 90-day cumulative contribution and repeat-purchase rate by acquisition channel, flagging cohorts whose economics deteriorate and incomplete 90-day windows. The script answers a narrower version: cohorts by acquisition month with total contribution, contribution per delivered order and repeat-customer rate, built two ways — one row at a time (Steps 1, 2 and 4) and one cohort per customer (Steps 3 and 5). Defect 7 lists what is not covered and Section 06 supplies a corrected query.

Step 1 — Acquisition cohort analysis

QUESTION 4 · STEP 1 — ACQUISITION COHORT ANALYSIS

```sql
-- Step 1: acquisition cohort analysis
USE case_study_1;

SELECT
  DATE_FORMAT(
    STR_TO_DATE(customer_acquisition_date, '%Y-%m-%d'),
    '%Y-%m'
  ) AS acquisition_month,

  COUNT(DISTINCT customer_id) AS total_customers,

  COUNT(DISTINCT CASE
    WHEN order_status = 'Delivered' THEN order_id
  END) AS delivered_orders,

  ROUND(SUM(
    CASE WHEN order_status = 'Delivered'
    THEN quantity * unit_selling_price - discount_amount
    ELSE 0 END
  ), 2) AS delivered_gmv,

  ROUND(SUM(
    CASE WHEN order_status = 'Delivered'
    THEN (quantity * unit_selling_price - discount_amount)
         - refund_amount - payment_fee - fulfilment_cost
    ELSE 0 END
  ), 2) AS contribution_proxy
FROM marketplace_transactions
GROUP BY acquisition_month
ORDER BY acquisition_month;
```

OUTPUT — FIRST 29 COHORTS PRINTED (2023-01 TO 2025-05)

acquisition_month	total_customers	delivered_orders	delivered_gmv	contribution_proxy
2023-01	32	22	19,901.59	17,801.42
2023-02	17	15	10,984.81	9,679.70
2023-03	19	15	9,444.51	8,185.37
2023-04	18	16	12,945.99	11,492.62
2023-05	26	21	9,378.86	7,897.85
2023-06	26	23	18,084.71	16,070.61
2023-07	27	23	9,909.24	8,069.97
2023-08	26	22	9,172.27	7,696.14
2023-09	26	19	13,845.84	12,187.09
2023-10	36	32	24,880.71	22,391.62
2023-11	17	13	8,663.99	7,697.44
2023-12	32	26	21,757.27	19,323.40
2024-01	25	26	20,462.24	18,153.42
2024-02	18	13	7,927.50	6,656.04
2024-03	24	20	14,521.74	12,812.71
2024-04	17	12	6,076.21	5,204.59
2024-05	18	16	9,108.63	7,836.96
2024-06	27	20	10,392.01	8,882.97
2024-07	23	21	7,768.68	6,221.05
2024-08	23	20	10,813.62	9,346.71
2024-09	24	20	12,607.26	11,026.45
2024-10	26	26	14,494.20	12,500.67
2024-11	25	22	11,973.80	10,504.81
2024-12	25	20	13,983.92	12,573.25

acquisition_month	total_customers	delivered_orders	delivered_gmv	contribution_proxy
2025-01	19	18	12,205.10	10,809.12
2025-02	31	22	12,394.85	10,725.78
2025-03	22	18	6,444.80	5,043.46
2025-04	28	24	11,627.14	9,808.35
2025-05	29	26	19,149.30	17,134.62

Step 2 — Contribution per delivered order

QUESTION 4 · STEP 2 — ACQUISITION COHORT CONTRIBUTION PER DELIVERED ORDER

```sql
-- Step 2: acquisition cohort contribution per delivered order
SELECT
  DATE_FORMAT(
    STR_TO_DATE(customer_acquisition_date, '%Y-%m-%d'),
    '%Y-%m'
  ) AS acquisition_month,
  COUNT(DISTINCT customer_id) AS total_customers,
  COUNT(DISTINCT CASE
    WHEN order_status = 'Delivered' THEN order_id
  END) AS delivered_orders,
  ROUND(
    SUM(
      CASE WHEN order_status = 'Delivered'
      THEN (quantity * unit_selling_price - discount_amount)
           - refund_amount - payment_fee - fulfilment_cost
      ELSE 0 END
    )
    / NULLIF(
      COUNT(DISTINCT CASE
        WHEN order_status = 'Delivered' THEN order_id
      END), 0
    ),
    2
  ) AS contribution_per_delivered_order
FROM marketplace_transactions
GROUP BY acquisition_month
ORDER BY contribution_per_delivered_order DESC;
```

OUTPUT — TOP 29 OF 35 COHORTS (FIRST 29 ROWS PRINTED)

acquisition_month	total_customers	delivered_orders	contribution_per_delivered_order
2023-01	32	22	809.16
2025-09	22	21	747.80
2023-12	32	26	743.21
2023-04	18	16	718.29
2023-10	36	32	699.74
2023-06	26	23	698.72
2024-01	25	26	698.21
2025-05	29	26	659.02
2025-11	21	14	648.26
2023-02	17	15	645.31
2023-09	26	19	641.43
2024-03	24	20	640.64
2024-12	25	20	628.66
2025-01	19	18	600.51
2023-11	17	13	592.11
2025-07	29	25	554.37
2024-09	24	20	551.32
2023-03	19	15	545.69

acquisition_month	total_customers	delivered_orders	contribution_per_delivered_order
2024-02	18	13	512.00
2024-05	18	16	489.81
2025-02	31	22	487.54
2024-10	26	26	480.80
2024-11	25	22	477.49
2025-08	22	19	471.76
2024-08	23	20	467.34
2025-10	25	25	460.12
2024-06	27	20	444.15
2024-04	17	12	433.72
2025-06	21	18	430.76

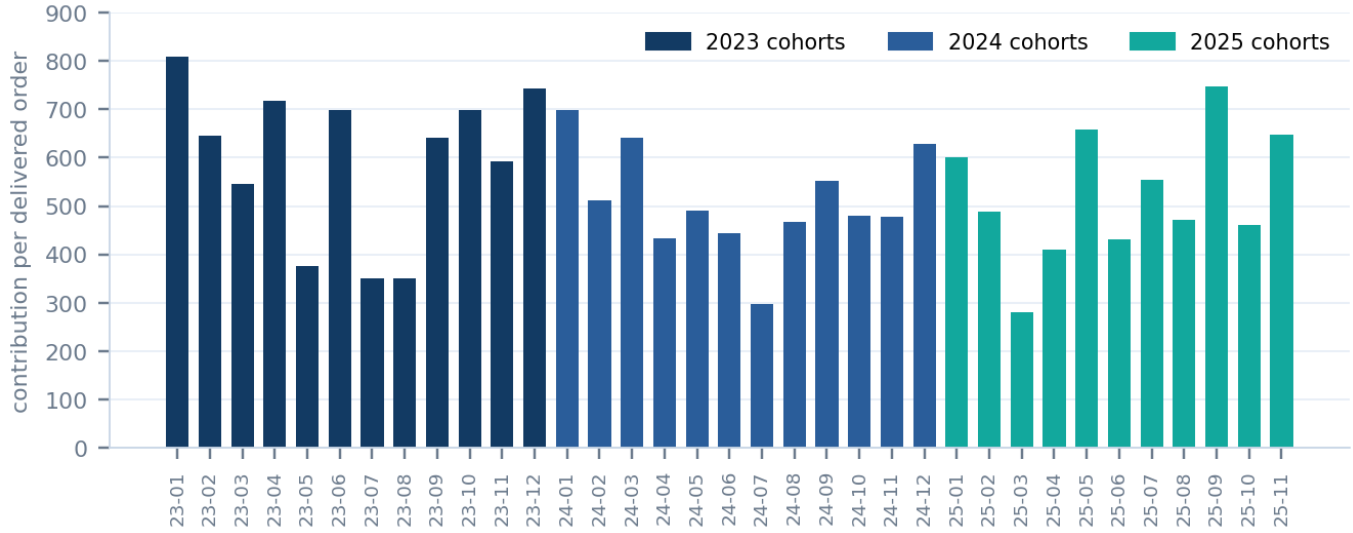


Figure 5.4 — Contribution per delivered order by acquisition cohort (row-level cohorts). Six cohorts not printed in Step 2 are derived ◆ from Steps 1 and 4.

Step 3 — Repeat-customer rate by cohort

QUESTION 4 · STEP 3 — REPEAT-CUSTOMER RATE BY ACQUISITION COHORT

```sql
-- Step 3: repeat-customer rate by acquisition cohort
SELECT
  acquisition_month,
  COUNT(*) AS total_customers,
  SUM(CASE
    WHEN delivered_orders >= 2 THEN 1
    ELSE 0
  END) AS repeat_customers,
  ROUND(
    SUM(CASE
      WHEN delivered_orders >= 2 THEN 1
      ELSE 0
    END) * 100.0 / COUNT(*),
    2
  ) AS repeat_customer_rate_pct
FROM (
  SELECT
    customer_id,
    DATE_FORMAT(
      STR_TO_DATE(
        MIN(customer_acquisition_date),
        '%Y-%m-%d'
      ),
      '%Y-%m'
    ) AS acquisition_month,
    COUNT(DISTINCT CASE
      WHEN order_status = 'Delivered' THEN order_id
    END) AS delivered_orders
```

```
  FROM marketplace_transactions
  GROUP BY customer_id
) AS customer_cohorts
GROUP BY acquisition_month
ORDER BY acquisition_month;
```

OUTPUT — FIRST 29 COHORTS PRINTED (2023-01 TO 2025-05)

acquisition_month	total_customers	repeat_customers	repeat_customer_rate_pct
2023-01	32	30	93.75
2023-02	13	13	100.00
2023-03	18	12	66.67
2023-04	12	11	91.67
2023-05	19	18	94.74
2023-06	16	15	93.75
2023-07	11	9	81.82
2023-08	14	13	92.86
2023-09	12	11	91.67
2023-10	12	5	41.67
2023-11	6	5	83.33
2023-12	7	7	100.00
2024-01	7	6	85.71
2024-02	7	3	42.86
2024-03	6	5	83.33
2024-04	1	1	100.00
2024-05	4	4	100.00
2024-06	5	4	80.00
2024-07	3	3	100.00
2024-08	2	2	100.00
2024-09	2	1	50.00
2024-10	3	2	66.67
2024-11	3	2	66.67
2024-12	3	2	66.67
2025-01	2	2	100.00
2025-02	1	0	0.00
2025-03	2	0	0.00
2025-04	5	2	40.00
2025-05	2	2	100.00

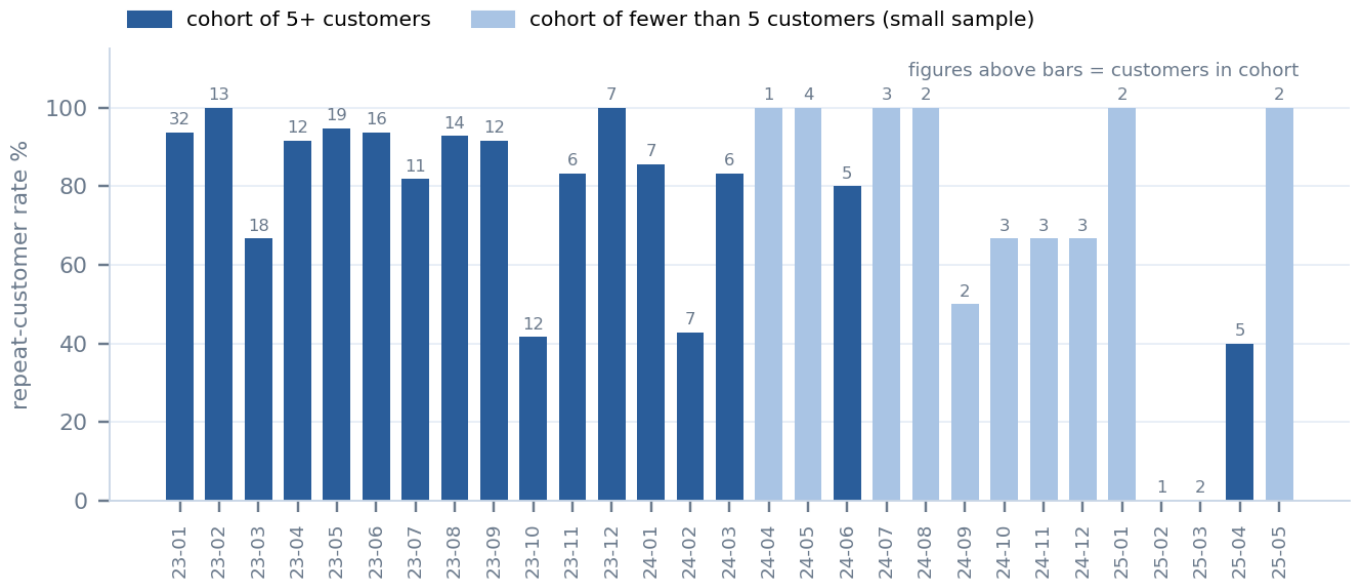


Figure 5.5 — Repeat-customer rate by cohort (one cohort per customer). Light bars mark cohorts of fewer than five customers, where a rate of 0% or 100% is noise.

Step 4 — Highest total and per-order contribution

QUESTION 4 · STEP 4 — COHORTS WITH THE HIGHEST TOTAL AND PER-ORDER CONTRIBUTION

```sql
-- Step 4: cohorts with the highest total and per-order contribution
SELECT
  DATE_FORMAT(
    STR_TO_DATE(customer_acquisition_date, '%Y-%m-%d'),
    '%Y-%m'
  ) AS acquisition_month,
  COUNT(DISTINCT customer_id) AS total_customers,
  COUNT(DISTINCT CASE
    WHEN order_status = 'Delivered'
      THEN order_id
  END) AS delivered_orders,
  ROUND(
    SUM(CASE
      WHEN order_status = 'Delivered'
        THEN (quantity * unit_selling_price - discount_amount)
             - refund_amount
             - payment_fee
             - fulfilment_cost
      ELSE 0
    END),
    2
  ) AS total_contribution,
  ROUND(
    SUM(CASE
      WHEN order_status = 'Delivered'
        THEN (quantity * unit_selling_price - discount_amount)
             - refund_amount
             - payment_fee
             - fulfilment_cost
      ELSE 0
    END)
    / NULLIF(
      COUNT(DISTINCT CASE
        WHEN order_status = 'Delivered'
          THEN order_id
      END),
      0
    ),
    2
  ) AS contribution_per_order
FROM marketplace_transactions
GROUP BY acquisition_month
ORDER BY total_contribution DESC;
```

OUTPUT — TOP 29 OF 35 COHORTS (FIRST 29 ROWS PRINTED)

acquisition_month	total_customers	delivered_orders	total_contribution	contribution_per_order
2023-10	36	32	22,391.62	699.74
2023-12	32	26	19,323.40	743.21

acquisition_month	total_customers	delivered_orders	total_contribution	contribution_per_order
2024-01	25	26	18,153.42	698.21
2023-01	32	22	17,801.42	809.16
2025-05	29	26	17,134.62	659.02
2023-06	26	23	16,070.61	698.72
2025-09	22	21	15,703.82	747.80
2025-07	29	25	13,859.16	554.37
2024-03	24	20	12,812.71	640.64
2024-12	25	20	12,573.25	628.66
2024-10	26	26	12,500.67	480.80
2023-09	26	19	12,187.09	641.43
2025-10	25	25	11,502.99	460.12
2023-04	18	16	11,492.62	718.29
2024-09	24	20	11,026.45	551.32
2025-01	19	18	10,809.12	600.51
2025-02	31	22	10,725.78	487.54
2024-11	25	22	10,504.81	477.49
2025-04	28	24	9,808.35	408.68
2023-02	17	15	9,679.70	645.31
2024-08	23	20	9,346.71	467.34
2025-11	21	14	9,075.62	648.26
2025-08	22	19	8,963.46	471.76
2024-06	27	20	8,882.97	444.15
2023-03	19	15	8,185.37	545.69
2023-07	27	23	8,069.97	350.87
2023-05	26	21	7,897.85	376.09
2024-05	18	16	7,836.96	489.81
2025-06	21	18	7,753.68	430.76

Step 5 — Cohort quality, one cohort per customer

QUESTION 4 · STEP 5 — COHORT QUALITY WITH ONE COHORT PER CUSTOMER

```sql
-- Step 5: cohort quality with one cohort per customer
SELECT
  acquisition_month,
  COUNT(*) AS total_customers,
  SUM(CASE
    WHEN delivered_orders >= 2 THEN 1
    ELSE 0
  END) AS repeat_customers,
  ROUND(
    100.0 * SUM(CASE
      WHEN delivered_orders >= 2 THEN 1
      ELSE 0
    END) / COUNT(*),
    2
  ) AS repeat_customer_rate_pct,
  SUM(delivered_orders) AS delivered_orders,
  ROUND(SUM(total_contribution), 2) AS total_contribution,
  ROUND(
    SUM(total_contribution) /
    NULLIF(SUM(delivered_orders), 0),
    2
  ) AS contribution_per_order
FROM (
  SELECT
    customer_id,
    DATE_FORMAT(
```

```
      STR_TO_DATE(
        MIN(customer_acquisition_date),
        '%Y-%m-%d'
      ),
      '%Y-%m'
    ) AS acquisition_month,
    COUNT(DISTINCT CASE
      WHEN order_status = 'Delivered'
        THEN order_id
    END) AS delivered_orders,
    SUM(CASE
      WHEN order_status = 'Delivered'
        THEN (quantity * unit_selling_price - discount_amount)
             - refund_amount
             - payment_fee
             - fulfilment_cost
      ELSE 0
    END) AS total_contribution
  FROM marketplace_transactions
  GROUP BY customer_id
) AS customer_cohorts
GROUP BY acquisition_month
ORDER BY total_contribution DESC;
```

OUTPUT — TOP 29 COHORTS BY CONTRIBUTION (FIRST 29 ROWS PRINTED)

acquisition_month	total_customers	repeat_customers	repeat_customer_rate_pct	delivered_orders	total_contribution	contribution_per_order
2023-01	32	30	93.75	129	70,938.39	549.91
2023-06	16	15	93.75	67	37,601.47	561.22
2023-05	19	18	94.74	64	36,674.16	573.03
2023-02	13	13	100.00	49	29,066.35	593.19
2023-04	12	11	91.67	43	23,477.08	545.98
2023-03	18	12	66.67	51	22,887.38	448.77
2023-10	12	5	41.67	32	22,832.05	713.50
2023-09	12	11	91.67	52	21,991.30	422.91
2023-08	14	13	92.86	42	21,337.77	508.04
2023-12	7	7	100.00	22	15,571.12	707.78
2024-01	7	6	85.71	21	12,837.81	611.32
2023-07	11	9	81.82	30	11,304.56	376.82
2024-05	4	4	100.00	10	10,311.53	1,031.15
2024-03	6	5	83.33	19	9,034.13	475.48
2023-11	6	5	83.33	13	8,928.39	686.80
2024-02	7	3	42.86	10	5,933.75	593.38
2025-04	5	2	40.00	7	5,623.33	803.33
2024-06	5	4	80.00	10	4,625.17	462.52
2024-04	1	1	100.00	3	2,950.39	983.46
2024-07	3	3	100.00	6	2,616.15	436.02
2024-12	3	2	66.67	7	2,499.86	357.12
2024-10	3	2	66.67	5	2,371.10	474.22
2025-01	2	2	100.00	5	2,089.22	417.84
2024-11	3	2	66.67	6	1,701.86	283.64
2025-05	2	2	100.00	5	1,215.55	243.11
2024-09	2	1	50.00	3	1,129.13	376.38
2025-07	3	1	33.33	2	930.43	465.22
2024-08	2	2	100.00	4	864.30	216.08
2025-06	2	0	0.00	1	587.35	587.35

KEY INSIGHT

Older cohorts carry the contribution. With one cohort per customer, the twelve 2023 cohorts hold 322,610.02 of contribution, 82.6% of the 390,592.92 total ◆, and the single cohort of January 2023 (32 customers, 129 delivered orders, 93.75% repeat) holds 70,938.39 (18.2%). Most 2023 cohorts repeat at 82–100%; the exceptions are 2023-03 (66.67%) and 2023-10 (41.67%), and the latter still earns 713.50 per order, the best of 2023 — fewer repeaters but higher-value orders. From mid-2024 the cohorts hold one to seven customers each, so their repeat rates (0%, 100%) and per-order values (for example 1,031.15 for 2024-05) are not reliable. **The two cohort methods disagree on size:** Step 1 lists 706 customer-cohort memberships across 29 cohorts while Step 3 finds only 230 customers across the same 29 ◆, because a customer's acquisition date is not constant across their rows (Defect 5). Step 5 is the sounder basis. Whether first-purchase economics are positive but 90-day economics deteriorate, and which windows are incomplete, cannot be read from these outputs; if the order data ends in December 2025, any customer first ordering after about 2 October 2025 has an incomplete 90-day window.

Question 5 — Discount-reduction break-even

Quantify current monthly contribution by category, calculate the delivered-GMV decline that would eliminate the benefit of a 10% reduction in discount expense, and identify the categories with the smallest tolerance for volume decline. The break-even is the discount saving divided by current contribution: the share of contribution that must be lost before the saving is cancelled out, assuming contribution scales in proportion to delivered volume.

Step 1 — Contribution and discount expense by category and month

QUESTION 5 · STEP 1 — CURRENT CONTRIBUTION AND DISCOUNT EXPENSE BY CATEGORY AND MON

```sql
-- Step 1: current contribution and discount expense by category and month
SELECT
  DATE_FORMAT(order_timestamp, '%Y-%m') AS month,
  product_category,

  ROUND(
    SUM(CASE WHEN order_status = 'Delivered'
      THEN discount_amount
      ELSE 0
    END) * 0.10,
    2
  ) AS discount_saving,

  ROUND(
    SUM(CASE
      WHEN order_status = 'Delivered'
      THEN (quantity * unit_selling_price - discount_amount)
           - refund_amount - payment_fee - fulfilment_cost
      ELSE 0
    END),
    2
  ) AS contribution_at_risk,

  ROUND(
    (
      SUM(CASE
        WHEN order_status = 'Delivered'
        THEN discount_amount
        ELSE 0
      END) * 0.10
    )
    /
    NULLIF(
      SUM(CASE
        WHEN order_status = 'Delivered'
        THEN (quantity * unit_selling_price - discount_amount)
             - refund_amount - payment_fee - fulfilment_cost
        ELSE 0
      END),
      0
    ) * 100,
    2
  ) AS break_even_gmv_decline_pct
FROM marketplace_transactions
GROUP BY
  DATE_FORMAT(order_timestamp, '%Y-%m'),
  product_category
ORDER BY
  month,
  break_even_gmv_decline_pct ASC;
```

OUTPUT — FIRST 29 OF 60 ROWS PRINTED (JANUARY – JUNE 2025)

month	product_category	discount_saving	contribution_at_risk	break_even_gmv_decline_pct
2025-01	Fashion	138.35	9,333.20	1.48
2025-01	Electronics	160.01	9,336.66	1.71
2025-01	Beauty	90.10	4,672.73	1.93
2025-01	Sports	113.44	5,368.54	2.11
2025-01	Home & Kitchen	196.15	7,755.91	2.53
2025-02	Sports	162.14	9,418.89	1.72
2025-02	Electronics	180.53	8,716.41	2.07
2025-02	Home & Kitchen	183.42	8,289.20	2.21
2025-02	Fashion	105.64	3,702.05	2.85
2025-02	Beauty	313.80	7,477.25	4.20
2025-03	Home & Kitchen	395.13	17,557.59	2.25
2025-03	Sports	226.58	8,066.84	2.81
2025-03	Fashion	192.37	6,372.60	3.02
2025-03	Electronics	87.19	2,738.47	3.18
2025-03	Beauty	326.65	8,686.32	3.76
2025-04	Sports	38.93	3,510.46	1.11
2025-04	Home & Kitchen	108.09	4,789.28	2.26
2025-04	Fashion	146.42	5,656.30	2.59
2025-04	Beauty	113.21	3,979.55	2.84
2025-04	Electronics	337.30	9,921.33	3.40
2025-05	Beauty	221.37	14,015.27	1.58
2025-05	Electronics	256.18	12,221.53	2.10
2025-05	Home & Kitchen	54.44	2,258.51	2.41
2025-05	Fashion	169.77	5,521.81	3.07
2025-05	Sports	75.23	2,270.56	3.31
2025-06	Electronics	26.89	2,210.92	1.22
2025-06	Home & Kitchen	74.18	5,512.99	1.35
2025-06	Sports	122.10	7,119.00	1.72
2025-06	Beauty	115.72	4,058.38	2.85

Step 2 — 10% discount-reduction scenario by category

QUESTION 5 · STEP 2 — 10% DISCOUNT-REDUCTION SCENARIO BY CATEGORY

```sql
-- Step 2: 10% discount-reduction scenario by category
SELECT
  product_category,
  ROUND(SUM(CASE
    WHEN order_status = 'Delivered'
    THEN discount_amount
    ELSE 0
  END) * 0.10, 2) AS discount_saving,

  ROUND(SUM(CASE
    WHEN order_status = 'Delivered'
    THEN (quantity * unit_selling_price - discount_amount)
         - refund_amount
         - payment_fee
         - fulfilment_cost
    ELSE 0
  END), 2) AS contribution_at_risk,
```

```
  ROUND(
    100.0 * (
      SUM(CASE
        WHEN order_status = 'Delivered'
        THEN discount_amount
        ELSE 0
      END) * 0.10
    ) / NULLIF(
      SUM(CASE
        WHEN order_status = 'Delivered'
        THEN (quantity * unit_selling_price - discount_amount)
             - refund_amount
             - payment_fee
             - fulfilment_cost
        ELSE 0
      END),
      0
    ),
    2
  ) AS break_even_gmv_decline_pct
FROM marketplace_transactions
GROUP BY product_category
ORDER BY break_even_gmv_decline_pct ASC;
```

OUTPUT — 5 ROWS

product_category	discount_saving	contribution_at_risk	break_even_gmv_decline_pct
Sports	1,288.46	61,702.04	2.09
Electronics	2,139.85	95,966.49	2.23
Fashion	1,796.34	63,546.35	2.83
Home & Kitchen	2,264.34	79,640.35	2.84
Beauty	2,705.58	89,737.69	3.01

Step 3 — Monthly contribution by category

QUESTION 5 · STEP 3 — MONTHLY CONTRIBUTION BY CATEGORY

```
-- Step 3: monthly contribution by category
SELECT
  DATE_FORMAT(order_timestamp, '%Y-%m') AS month,
  product_category,

  COUNT(DISTINCT CASE
    WHEN order_status = 'Delivered'
    THEN order_id
  END) AS delivered_orders,

  ROUND(SUM(CASE
    WHEN order_status = 'Delivered'
    THEN quantity * unit_selling_price - discount_amount
    ELSE 0
  END), 2) AS delivered_gmv,

  ROUND(SUM(CASE
    WHEN order_status = 'Delivered'
    THEN discount_amount
    ELSE 0 END), 2) AS discount_expense,

  ROUND(SUM(CASE
    WHEN order_status = 'Delivered'
    THEN (quantity * unit_selling_price - discount_amount)
         - refund_amount - payment_fee - fulfilment_cost
    ELSE 0
  END), 2) AS current_contribution
FROM marketplace_transactions
GROUP BY
  DATE_FORMAT(order_timestamp, '%Y-%m'),
  product_category
ORDER BY
  month,
  product_category;
```

OUTPUT — FIRST 29 OF 60 ROWS PRINTED (JANUARY – JUNE 2025)

month	product_category	delivered_orders	delivered_gmv	discount_expense	current_contribution
2025-01	Beauty	12	5,430.63	900.97	4,672.73

month	product_ category	delivered_ orders	delivered_ gmv	discount_ expense	current_ contribution
2025-01	Electronics	15	10,555.70	1,600.12	9,336.66
2025-01	Fashion	14	10,562.80	1,383.50	9,333.20
2025-01	Home & Kitchen	13	8,992.57	1,961.48	7,755.91
2025-01	Sports	7	6,080.60	1,134.45	5,368.54
2025-02	Beauty	13	8,464.12	3,137.95	7,477.25
2025-02	Electronics	11	9,654.48	1,805.29	8,716.41
2025-02	Fashion	8	4,278.22	1,056.45	3,702.05
2025-02	Home & Kitchen	20	9,743.69	1,834.17	8,289.20
2025-02	Sports	10	10,362.34	1,621.42	9,418.89
2025-03	Beauty	17	10,069.28	3,266.54	8,686.32
2025-03	Electronics	10	3,542.87	871.90	2,738.47
2025-03	Fashion	9	7,205.48	1,923.70	6,372.60
2025-03	Home & Kitchen	18	19,154.82	3,951.28	17,557.59
2025-03	Sports	14	9,213.70	2,265.82	8,066.84
2025-04	Beauty	11	4,593.28	1,132.15	3,979.55
2025-04	Electronics	16	11,120.80	3,372.99	9,921.33
2025-04	Fashion	11	6,585.47	1,464.18	5,656.30
2025-04	Home & Kitchen	10	5,545.62	1,080.91	4,789.28
2025-04	Sports	8	4,160.20	389.30	3,510.46
2025-05	Beauty	12	15,153.81	2,213.74	14,015.27
2025-05	Electronics	20	13,864.68	2,561.82	12,221.53
2025-05	Fashion	16	6,664.47	1,697.72	5,521.81
2025-05	Home & Kitchen	8	2,820.77	544.37	2,258.51
2025-05	Sports	7	2,741.15	752.30	2,270.56
2025-06	Beauty	10	4,854.59	1,157.16	4,058.38
2025-06	Electronics	4	2,510.15	268.90	2,210.92
2025-06	Fashion	13	5,491.59	1,426.84	4,510.71
2025-06	Home & Kitchen	12	6,538.77	741.82	5,512.99

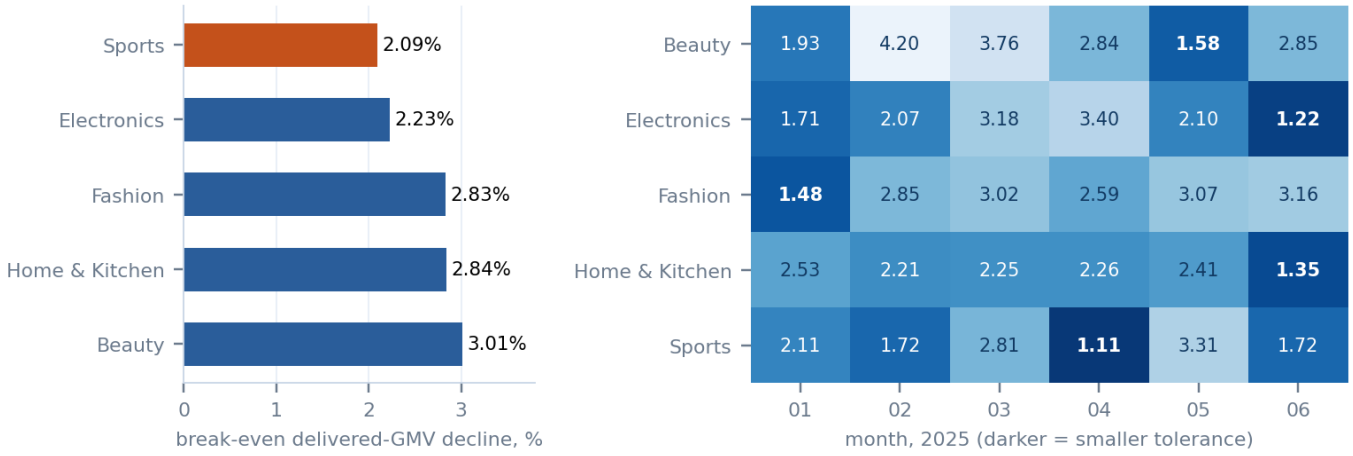


Figure 5.6 — Break-even delivered-GMV decline by category (full year, left) and by month (January–June, right; the June Fashion cell is derived ◆ from Step 3).

KEY INSIGHT

Sports has the smallest tolerance for volume decline. A 10% cut in its discount expense (saving 1,288.46) is cancelled by a delivered-volume fall of just 2.09%, followed by Electronics (2.23%), Fashion (2.83%), Home & Kitchen (2.84%) and Beauty (3.01%). Every category therefore tolerates a loss of only two to three percent of volume, so the cut pays off only if demand is very inelastic. By month the margin is thinner still: Sports in April needs only 1.11%, Electronics in June 1.22% and Home & Kitchen in June 1.35%, while Beauty in February reaches 4.20%. Because contribution is understated by Defect 1, the true break-evens are lower again (Section 06).

SECTION 06

Code Review & Corrections

The script was checked against the data it printed. Seven logic and data defects and six design observations are documented below, each with its effect and a correction.

DEFECTS FOUND	HIGH SEVERITY	DESIGN OBSERVATIONS	DELIVERED GMV UNDERSTATED
7	4	6	≈22.8%
logic and data	Defects 1, 2, 4 and 5	no effect on printed numbers	Defect 1 ◆ · 447,907 → 549,853

HOW TO READ THIS SECTION

Every defect is supported by arithmetic on numbers the script itself printed. The corrected SQL has **not** been executed — the source database was not available — so corrected figures are marked ◆ as derived estimates and should be confirmed by re-running the corrected queries on your server.

Summary of Defects

#	Defect	Where	Severity	Effect
1	Discount deducted twice from GMV	All GMV and contribution queries	High	Delivered GMV and contribution understated by the delivered discount (≈22.8% / 26.1% ◆); false “negative” transactions
2	Refund leg of the proxy is always zero	Q1 Step 10, Q3 Steps 3–6	High	Refund burden prints 0.00%; return costs never reach the proxy; “High” risk rule unreachable
3	NTILE(100) bands are not 1%, 5%, 10%	Q2 Step 2	Medium	“Top 5%” is 15 customers (6.6%), “Top 10%” is 30 (13.3%); concentration overstated
4	order_id is reused across customers	Q1 order_count, Q3 Step 6, Q4	High	COUNT(DISTINCT order_id) undercounts orders; per-order metrics inflated
5	Customer attributes differ between a customer’s rows	Q3 Steps 6–7, Q4	High	Cohorts and segments double-count customers; row-level and customer-level cohorts disagree
6	Status buckets do not reconcile to total GMV	Q1 Steps 2–4	Medium	26,183 ◆ of GMV (4.7%) sits in a status the bridge never names
7	Requested filters and metrics not implemented	Q1, Q2 Step 3, Q3 Step 4, Q4	Medium	Months, percentile, high-value, channel and 90-day questions not answered by a query

Defect 1 — Discount is deducted twice

Evidence. `discount_amount` equals `quantity × (unit_list_price − unit_selling_price)` on all ten sample rows in Section 03, so the selling price is already net of discount. The ten weakest transactions in Question 3, Step 4 confirm it independently: for every row, list value − discount equals the printed delivered GMV plus the discount (T00354: 371.96 − 167.38 = 204.58 = 37.20 + 167.38). Every GMV expression in the script, `quantity * unit_selling_price - discount_amount`, therefore removes the discount a second time.

Correction. Use `quantity * unit_selling_price` — or, equivalently, `quantity * unit_list_price - discount_amount` — wherever GMV is calculated, and add `discount_amount` back in the contribution proxy.

CORRECTION · DEFECT 1 — GMV AND CONTRIBUTION WITHOUT THE DOUBLE DEDUCTION

```sql
-- GMV: selling price is already net of discount
SUM(quantity * unit_selling_price) AS gmv

-- Contribution proxy (delivered rows)
SUM(CASE WHEN order_status = 'Delivered'
    THEN quantity * unit_selling_price
         - refund_amount - payment_fee - fulfilment_cost
    ELSE 0 END) AS contribution_proxy
```

Estimated impact ◆

Measure	As reported	Corrected ◆	Change
Delivered GMV	447,907.10	549,852.80	+22.8%

Measure	As reported	Corrected ◆	Change
Contribution proxy	390,592.92	492,538.62	+26.1%
Contribution ÷ delivered GMV	87.20%	89.58%	+2.37 pts
Break-even decline · Sports	2.09	1.73	smaller tolerance
Break-even decline · Electronics	2.23	1.82	smaller tolerance
Break-even decline · Fashion	2.83	2.20	smaller tolerance
Break-even decline · Home & Kitchen	2.84	2.21	smaller tolerance
Break-even decline · Beauty	3.01	2.32	smaller tolerance

Corrected totals add the delivered discount (10 × the five discount_saving values in Question 5, Step 2 = 101,945.70) to the reported totals, which holds only if the identity above applies to every delivered row — it did on all twenty rows checked. The ranking of categories by break-even does not change.

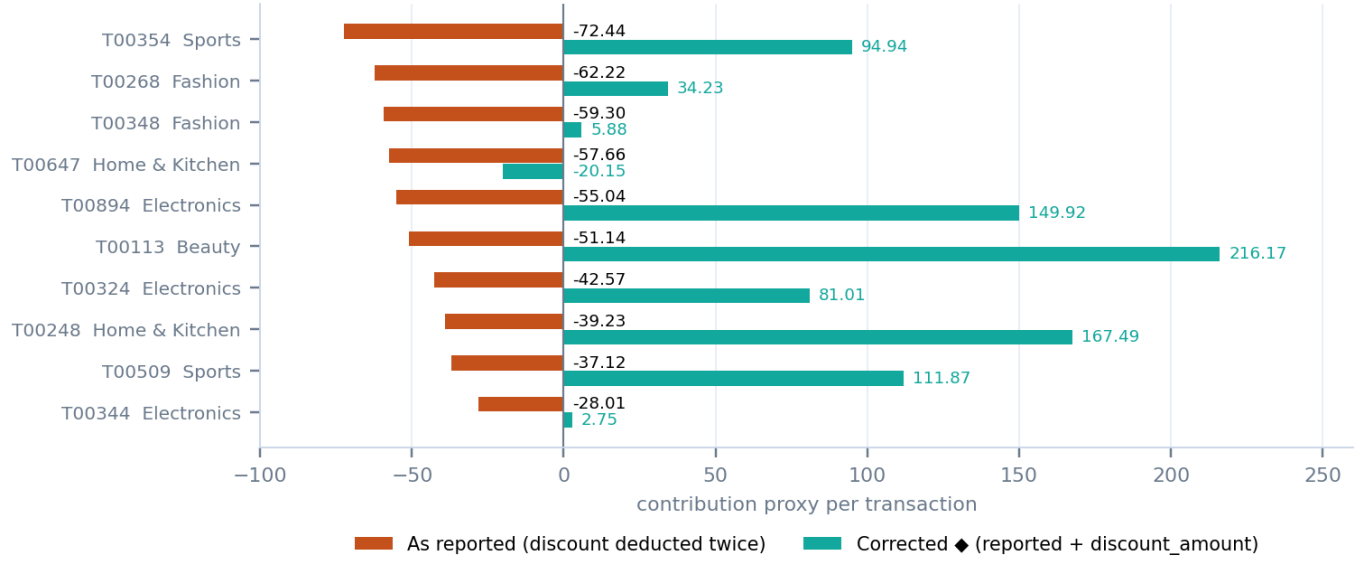


Figure 6.1 — The ten weakest delivered transactions of Question 3, Step 4, as reported and ◆ corrected by adding back the discount. Nine of ten turn positive; T00647 stays negative.

Defect 2 — The refund leg of the contribution proxy is always zero

Evidence. Refunds are recorded only on returned orders (T00003: Returned, refund 230.58; every delivered and cancelled sample row has 0), but the proxy subtracts `refund_amount` only on Delivered rows. January's proxy confirms it: 41,622.30 − 1,232.67 − 3,922.59 = 36,467.04, with no refund deduction, and the identity holds in all twelve months ◆. Consequently refund_rate_pct prints 0 for every category and segment, while the bridge shows 37,945.29 of refunds across the year (8,236.48 in June alone). Returned orders also incur payment and fulfilment cost that the proxy ignores.

Correction. Measure refund burden on delivered and returned orders together and charge the fees and fulfilment cost of returned orders to the bridge as return leakage. (Subtracting the refund again from delivered contribution would double-count, because returned GMV is already excluded.)

CORRECTION · DEFECT 2 — REFUND BURDEN AND RETURN LEAKAGE BY MONTH

```sql
SELECT
  DATE_FORMAT(STR_TO_DATE(order_timestamp, '%Y-%m-%d'), '%Y-%m') AS order_month,
  ROUND(100 * SUM(CASE WHEN order_status IN ('Delivered', 'Returned')
                  THEN refund_amount ELSE 0 END)
        / NULLIF(SUM(CASE WHEN order_status IN ('Delivered', 'Returned')
                     THEN quantity * unit_selling_price ELSE 0 END), 0), 2)
    AS refund_burden_pct,
  ROUND(SUM(CASE WHEN order_status = 'Returned'
                 THEN payment_fee + fulfilment_cost ELSE 0 END), 2)
    AS return_cost_leakage
FROM marketplace_transactions
GROUP BY order_month
ORDER BY order_month;
```

Defect 3 — NTILE(100) does not produce 1%, 5% and 10% bands

Evidence. 226 customers divide into 100 buckets as 26 buckets of three customers and 74 of two. “Top 1%” is therefore 3 customers (1.3% of customers) — the sum of C0100, C0097 and C0002 is exactly the printed 22,706.82 — “Top 5%” is 15 customers (6.6%) and “Top 10%” is 30 (13.3%).

Band	Reported (customers)	GMV share · contribution share	Corrected ◆ (customers)	GMV share · contribution share
Top 1%	3	5.07% · 5.41%	3 (CEIL of 2.26)	5.07% · 5.41%
Top 5%	15	20.10% · 21.14%	12 (CEIL of 11.3)	16.72% · 17.66%
Top 10%	30	34.10% · 35.62%	23 (CEIL of 22.6)	28.22% · 29.61%

Corrected shares are recomputed from the 29 customers printed in Question 2, Step 1. The conclusion is unchanged — contribution share still exceeds GMV share — but concentration is lower than reported.

CORRECTION · DEFECT 3 — EXACT PERCENTILE BANDS

```sql
WITH ranked AS (
  SELECT customer_id, delivered_gmv, contribution_proxy,
         ROW_NUMBER() OVER (ORDER BY delivered_gmv DESC) AS rn,
         COUNT(*) OVER () AS n
  FROM customer_totals
)
SELECT b.band,
       ROUND(SUM(r.delivered_gmv), 2) AS delivered_gmv,
       ROUND(SUM(r.contribution_proxy), 2) AS contribution_proxy
FROM ranked r
JOIN (SELECT 'Top 1%' AS band, 0.01 AS p
      UNION ALL SELECT 'Top 5%', 0.05
      UNION ALL SELECT 'Top 10%', 0.10) b
  ON r.rn <= CEIL(b.p * r.n)
GROUP BY b.band;
```

Defect 4 — order_id is reused across customers

Evidence. Question 3, Step 6 counts distinct delivered order_ids per segment; Step 7 counts them per customer. If order_id identified one customer’s order the two would agree. They do not: Standard shows 333 orders in Step 6 against about 457 implied by Step 7 (204 customers × 2.24), Premium 175 against about 198, High Value 67 against about 68 ◆. The same order_id therefore appears under different customers, so `COUNT(DISTINCT order_id)` undercounts orders and every per-order metric (contribution per order, average order value, monthly order_count) is inflated.

Correction. Count an order as the pair customer_id + order_id, or count transaction_id if each line is an order, after checking how often order_id repeats.

CORRECTION · DEFECT 4 — DIAGNOSE AND COUNT ORDERS CORRECTLY

```sql
-- How many order_ids belong to more than one customer?
SELECT COUNT(*) AS shared_order_ids
FROM (SELECT order_id
      FROM marketplace_transactions
      GROUP BY order_id
      HAVING COUNT(DISTINCT customer_id) > 1) x;

-- Count orders as customer + order
COUNT(DISTINCT customer_id, order_id) AS order_count
```

Defect 5 — Customer attributes are not stable per customer

Evidence. Customer-level facts are repeated on every row and differ between a customer’s rows. Segments: Step 7 returns 394 customer-segment pairs for the 226 customers who bought. Acquisition dates: Question 4, Step 1 lists 706 customer-cohort memberships across 29 cohorts, while Step 3 (earliest date per customer) finds 230 customers across the same cohorts ◆; 2023-10 holds 36 customers by row and 12 by customer. Segment and cohort results are therefore sensitive to how the duplicates are resolved, and the “High Value” label, which lands on the weakest customers, is suspect.

Correction. Resolve each customer to one acquisition date, channel and segment before analysing, and keep them in a customer dimension.

CORRECTION · DEFECT 5 — DIAGNOSE AND RESOLVE CUSTOMER ATTRIBUTES

```sql
-- Customers whose attributes change between rows
SELECT customer_id,
       COUNT(DISTINCT customer_acquisition_date) AS acquisition_dates,
       COUNT(DISTINCT acquisition_channel)       AS channels,
       COUNT(DISTINCT customer_segment)          AS segments
FROM marketplace_transactions
GROUP BY customer_id
HAVING acquisition_dates > 1 OR channels > 1 OR segments > 1;

-- One acquisition date per customer
SELECT customer_id,
       MIN(STR_TO_DATE(customer_acquisition_date, '%Y-%m-%d')) AS acquisition_date
FROM marketplace_transactions
GROUP BY customer_id;
```

Defect 6 — The status buckets do not reconcile to total GMV

Evidence. Delivered, cancelled and returned GMV do not add up to total GMV in any month. The residual ◆, orders in a status the script never names, is 26,183.43 for the year (4.7% of GMV); in January alone it is 1,971.93.

Month	Total GMV	Delivered	Cancelled	Returned	Other statuses ◆	Other % of GMV
2025-01	50,340.60	41,622.30	5,288.67	1,457.70	1,971.93	3.9%
2025-02	46,456.72	42,502.85	676.13	3,211.55	66.19	0.1%
2025-03	61,712.60	49,186.15	6,686.20	4,438.13	1,402.12	2.3%
2025-04	37,012.57	32,005.37	1,363.19	1,159.53	2,484.48	6.7%
2025-05	47,467.78	41,244.88	1,080.45	4,051.34	1,091.11	2.3%
2025-06	42,177.12	27,704.86	3,939.08	10,124.94	408.24	1.0%
2025-07	54,038.29	47,015.45	2,029.46	3,345.50	1,647.88	3.0%
2025-08	40,984.74	34,999.66	3,014.34	2,316.36	654.38	1.6%
2025-09	47,570.73	38,529.06	4,267.44	3,732.00	1,042.23	2.2%
2025-10	42,816.36	24,875.36	4,499.51	5,778.27	7,663.22	17.9%
2025-11	36,213.83	27,903.66	1,314.99	5,671.41	1,323.77	3.7%
2025-12	54,526.66	40,317.50	3,446.44	4,334.84	6,427.88	11.8%
Year	561,318.00	447,907.10	37,605.90	49,621.57	26,183.43	4.7%

Correction. List the statuses present and add an explicit “other” column to the bridge.

CORRECTION · DEFECT 6 — WHICH STATUSES EXIST?

```sql
SELECT order_status,
       COUNT(*) AS lines,
       ROUND(SUM(quantity * unit_selling_price), 2) AS gmv
FROM marketplace_transactions
GROUP BY order_status
ORDER BY gmv DESC;
```

Defect 7 — Requested filters and metrics are not implemented

- **Question 1.** No query lists the months with positive GMV growth and negative contribution growth. The answer (none) was read from the growth columns.
- **Question 2, Step 3.** The filter keeps every customer with negative contribution and never applies the 90th percentile (`gmv_percentile >= 0.90`).
- **Question 3, Step 4.** Titled “high-value” but ordered only by contribution, so it surfaces low-value lines (delivered GMV of 27–76).
- **Question 4.** Does not compute first-purchase contribution, 90-day cumulative contribution or repeat rate by acquisition channel, and does not flag incomplete 90-day windows.

CORRECTION · DEFECT 7A — MONTHS WITH GMV UP AND CONTRIBUTION DOWN

```sql
WITH monthly AS (
  SELECT DATE_FORMAT(STR_TO_DATE(order_timestamp, '%Y-%m-%d'), '%Y-%m') AS order_month,
         SUM(quantity * unit_selling_price) AS gmv,
         SUM(CASE WHEN order_status = 'Delivered'
             THEN quantity * unit_selling_price
                  - refund_amount - payment_fee - fulfilment_cost
             ELSE 0 END) AS contribution
  FROM marketplace_transactions
  GROUP BY order_month
),
growth AS (
  SELECT order_month,
         100 * (gmv - LAG(gmv) OVER w) / NULLIF(LAG(gmv) OVER w, 0)
           AS gmv_growth_pct,
         100 * (contribution - LAG(contribution) OVER w)
           / NULLIF(LAG(contribution) OVER w, 0) AS contribution_growth_pct
  FROM monthly
  WINDOW w AS (ORDER BY order_month)
)
SELECT *
FROM growth
WHERE gmv_growth_pct > 0 AND contribution_growth_pct < 0;
```

CORRECTION · DEFECT 7B — HIGH-VALUE CUSTOMERS (90TH PERCENTILE) WITH NEGATIVE CONTRI

```sql
-- add to the Question 2, Step 3 final select
WHERE gmv_percentile >= 0.90
  AND contribution_proxy < 0
```

CORRECTION · DEFECT 7C — HIGH-VALUE TRANSACTIONS WITH WEAK ECONOMICS

```sql
WITH tx AS (
  SELECT transaction_id, customer_id, product_category,
         quantity * unit_list_price AS list_value,
         quantity * unit_selling_price
           - refund_amount - payment_fee - fulfilment_cost AS contribution,
         PERCENT_RANK() OVER (ORDER BY quantity * unit_list_price) AS value_pctl
  FROM marketplace_transactions
  WHERE order_status = 'Delivered'
)
SELECT *
FROM tx
WHERE value_pctl >= 0.90
ORDER BY contribution ASC
LIMIT 10;
```

CORRECTION · DEFECT 7D — COHORTS BY CHANNEL WITH 90-DAY CONTRIBUTION AND WINDOW FLA

```sql
WITH orders AS (
  SELECT customer_id, order_id,
         MIN(STR_TO_DATE(order_timestamp, '%Y-%m-%d')) AS order_date,
         SUM(quantity * unit_selling_price
             - refund_amount - payment_fee - fulfilment_cost) AS contribution
  FROM marketplace_transactions
  WHERE order_status = 'Delivered'
  GROUP BY customer_id, order_id
),
first_order AS (
  SELECT customer_id, MIN(order_date) AS first_date
  FROM orders GROUP BY customer_id
),
customer_dim AS (
  SELECT customer_id,
         MIN(STR_TO_DATE(customer_acquisition_date, '%Y-%m-%d')) AS acquisition_date
  FROM marketplace_transactions GROUP BY customer_id
),
channel_dim AS (  -- channel on the earliest-dated row
  SELECT t.customer_id, MIN(t.acquisition_channel) AS acquisition_channel
  FROM marketplace_transactions t
  JOIN customer_dim d
    ON t.customer_id = d.customer_id
   AND STR_TO_DATE(t.customer_acquisition_date, '%Y-%m-%d') = d.acquisition_date
  GROUP BY t.customer_id
),
data_end AS (
  SELECT MAX(STR_TO_DATE(order_timestamp, '%Y-%m-%d')) AS last_date
  FROM marketplace_transactions
),
cohort AS (
  SELECT d.customer_id,
         DATE_FORMAT(d.acquisition_date, '%Y-%m') AS acquisition_month,
         ch.acquisition_channel, f.first_date,
```

```
        SUM(CASE WHEN o.order_date = f.first_date
            THEN o.contribution ELSE 0 END) AS first_contribution,
        SUM(CASE WHEN o.order_date < DATE_ADD(f.first_date, INTERVAL 90 DAY)
            THEN o.contribution ELSE 0 END) AS contribution_90d,
        COUNT(DISTINCT CASE
            WHEN o.order_date < DATE_ADD(f.first_date, INTERVAL 90 DAY)
            THEN o.order_id END) AS orders_90d
  FROM customer_dim d
  JOIN channel_dim ch ON ch.customer_id = d.customer_id
  JOIN first_order f  ON f.customer_id = d.customer_id
  JOIN orders o       ON o.customer_id = d.customer_id
  GROUP BY d.customer_id, acquisition_month, ch.acquisition_channel, f.first_date
)
SELECT acquisition_month, acquisition_channel,
       COUNT(*) AS customers,
       ROUND(AVG(first_contribution), 2) AS avg_first_purchase_contribution,
       ROUND(AVG(contribution_90d), 2)   AS avg_90d_contribution,
       ROUND(100 * AVG(orders_90d >= 2), 2) AS repeat_rate_90d_pct,
       SUM(DATE_ADD(first_date, INTERVAL 90 DAY) > (SELECT last_date FROM data_end))
         AS incomplete_window_customers,
       CASE WHEN AVG(first_contribution) > 0
            AND AVG(contribution_90d) < AVG(first_contribution)
           THEN 'Deteriorates' ELSE 'Holds' END AS ninety_day_trend
FROM cohort
GROUP BY acquisition_month, acquisition_channel
ORDER BY acquisition_month, acquisition_channel;
```

Design Observations

#	Observation	Suggestion
A	**The proxy is GMV-based, and commission is unused.** Seller commission (55,751.85) is calculated in the bridge but never enters the proxy, which treats GMV as the marketplace's revenue and so shows a margin near 87%. On a take-rate view commission is the revenue; commission less payment fees and fulfilment cost is -1,562.33 ◆ (before Defect 1), if the marketplace bears fulfilment.	Define the proxy explicitly as a take-rate or GMV view and state who bears each cost.
B	**Costs only on delivered rows.** Payment fees and fulfilment cost of cancelled and returned orders are excluded from the bridge.	Report them as leakage lines (see Defect 2).
C	**Risk labels use arbitrary thresholds.** The commercial-risk CASE (16%, 15%, 5%, 4%) is not derived from the data, and its refund leg cannot trigger (Defect 2).	Set thresholds from percentiles or business policy and document them.
D	**Break-even assumes proportional loss.** Dividing the saving by total contribution treats contribution as scaling linearly with volume, and uses delivered-row discounts while Question 3 discount rates use all statuses.	Use incremental margin per unit and a single status scope.
E	**Date and documentation hygiene.** Question 5 formats the text date directly (DATE_FORMAT(order_timestamp, ...)) while other queries use STR_TO_DATE; the capture under Question 3, Step 8 is Step 7's table; Question 4 and 5 captures show only 29 rows; one comment is in mixed Hindi–English (normalised here).	Convert dates to DATE once, keep one date idiom, and export complete results.
F	**One denormalised table.** Customer, seller and order attributes are repeated on every line, which allowed Defects 4 and 5.	Add customer, order and seller dimensions with keys, DATE columns and indexes.

SECTION 07

Results, Setup & Roadmap

Headline results, how to run the case study, and what to build next.

Headline Results

Question	Headline result (as the script printed it)
Q1 Bridge	Total GMV 561,318.00 → delivered 447,907.10 → contribution proxy 390,592.92 ◆. No month combines rising GMV with falling contribution; February and November show the reverse.
Q2 Concentration	Top 3 / 15 / 30 customers hold 5.07% / 20.10% / 34.10% of delivered GMV and 5.41% / 21.14% / 35.62% of contribution. None of 226 customers is loss-making; the biggest are also the most profitable.
Q3 Discounts	Discount rates run 13.22% (Sports) to 16.53% (Beauty) and 14.04% (Social) to 17.06% (Affiliate). Discount rate alone does not explain contribution per order. “High Value” customers are the weakest segment on order value, contribution and repeat rate.
Q4 Cohorts	2023 cohorts generate 82.6% of contribution ◆; January 2023 alone 70,938.39. Later cohorts are too small for reliable repeat rates. 90-day and channel questions are not answered.
Q5 Break-even	A 10% discount cut is cancelled by a 2.09% (Sports) to 3.01% (Beauty) fall in delivered volume; monthly values reach as low as 1.11%.
Review	Seven defects; the largest is the double-counted discount, which understates delivered GMV by about 22.8% ◆.

How to Run

- **1.** Load `marketplace_transactions` into a database named `case_study_1` on MySQL 8 or later (window functions are required).
- **2.** Run Section 03, then Questions 1–5 in order; each step is self-contained apart from the CTEs it defines.
- **3.** Apply the corrections in Section 06 and re-run to replace the ◆ estimates with executed figures.

Object Inventory

Object	Count	Detail
Database	1	case_study_1
Table	1	marketplace_transactions (18 columns)
Analytical questions	5	Q1 to Q5
Analytical queries	32	Q1 11 · Q2 4 · Q3 9 · Q4 5 · Q5 3
Setup statements	4	CREATE DATABASE, USE, SHOW TABLES, DESCRIBE
Charts in this document	12	figures 2.1 to 6.1
Corrected queries	9	Section 06

Roadmap

- **Correct and re-run.** Fix the GMV expression, return costs and order counting, then replace every ◆ figure with an executed one.
- **Normalise the model.** Customer, order and seller dimension tables with primary and foreign keys, DATE columns and indexes on customer_id, order_status and order date.
- **Take-rate P&L.** Add a view in which seller commission is revenue and payment, fulfilment and return costs are charged to it.
- **Proper cohorts.** Run the 90-day, channel-level cohort query with an incomplete-window flag and a minimum cohort size.
- **Elasticity scenarios.** Replace the break-even with a table of volume-loss scenarios using incremental margin per category.
- **Parameterise.** Turn the 10% discount cut, percentile bands and risk thresholds into variables or a small assumptions table.